



# โครงสร้างข้อมูลกราฟและการสำรวจกราฟ

## (Graph data structure and Graph Exploration)

อ.ลือพล พิพานเมฆาภรณ์

[luepol.p@sci.kmutnb.ac.th](mailto:luepol.p@sci.kmutnb.ac.th)

ภาควิชาวิทยาการคอมพิวเตอร์และสารสนเทศ

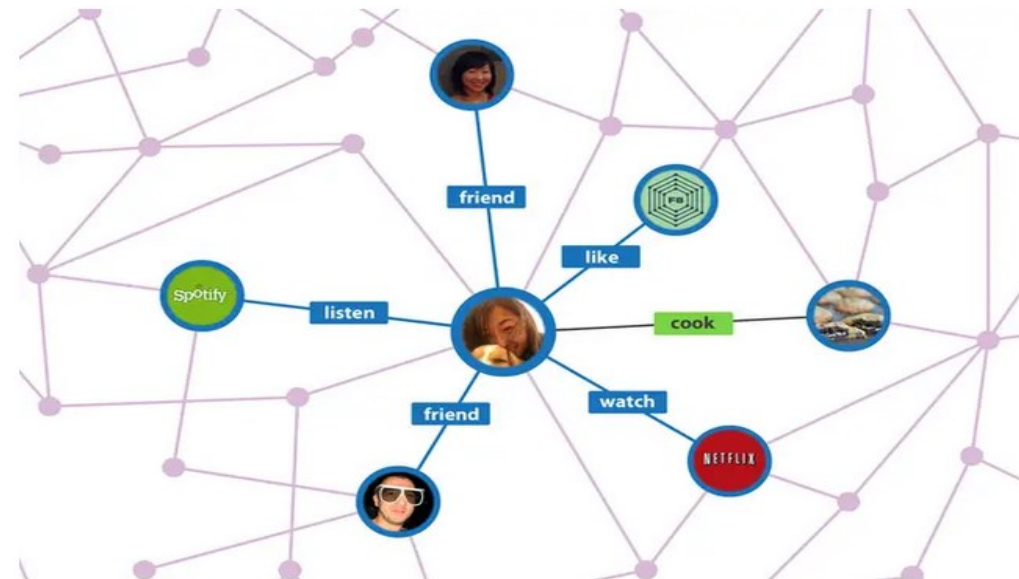
มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ

- โครงสร้างข้อมูลกราฟ และ Graph Implementation
  - เมตริกซ์ประชิด (adjacency matrix)
  - ลิสต์ประชิด (adjacency list)
- อัลกอริทึมสำรวจกราฟ (Graph exploration)
  - Breadth First Search (BFS)
  - Depth First Search (DFS)
- แอปพลิเคชันของ BFS / DFS
  - การหา shortest path ในกราฟ
  - การตรวจสอบวงวน (Cycle checking) ในกราฟทิศทาง
  - Connected components ในกราฟ
- การหาความสัมพันธ์ระหว่างเวอร์เท็กซ์ในกราฟ
  - Warshall's algorithm

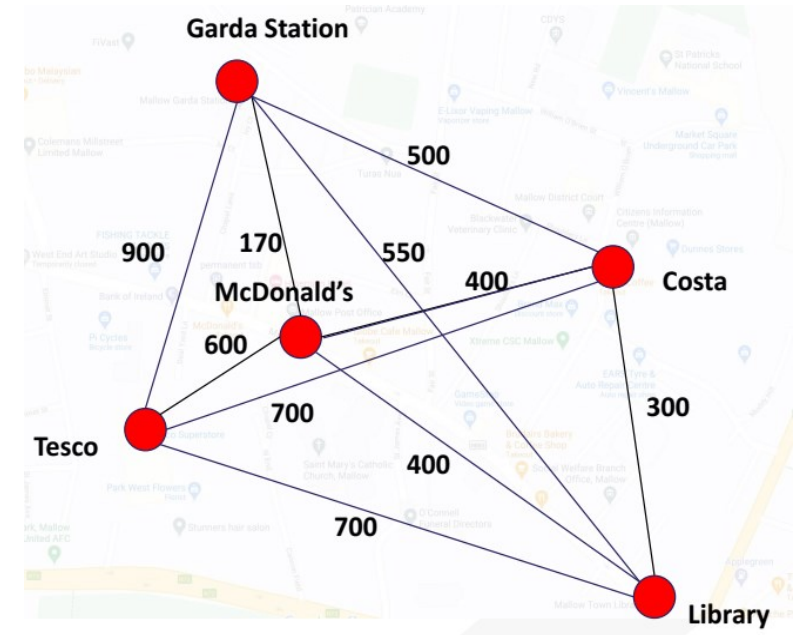
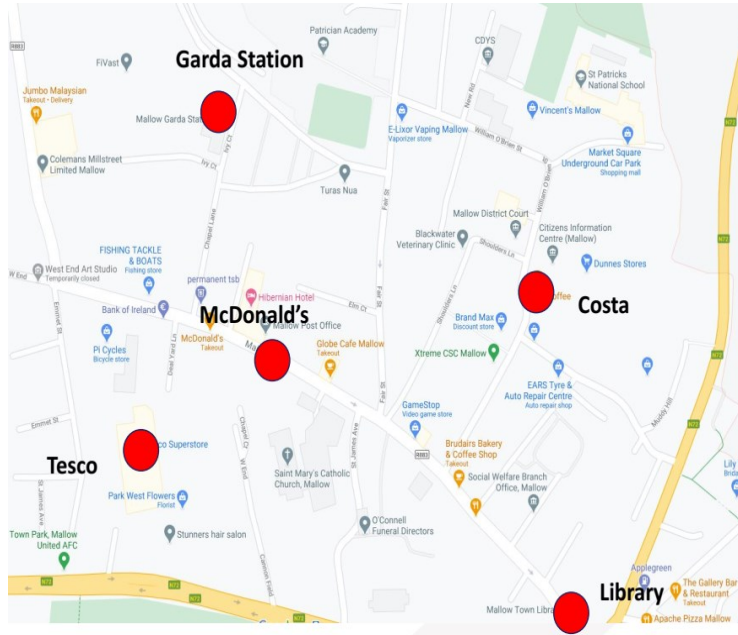
# กราฟ (graph)

- กราฟ (graph) เป็นโครงสร้างข้อมูลที่ใช้แสดงความสัมพันธ์ระหว่างข้อมูลหรือเอนทิตี (entity) ที่สนใจ

- กราฟประกอบไปด้วย
  - เซตเวอร์เท็กซ์ (vertex)
  - เซตของเอจด์ (edge)



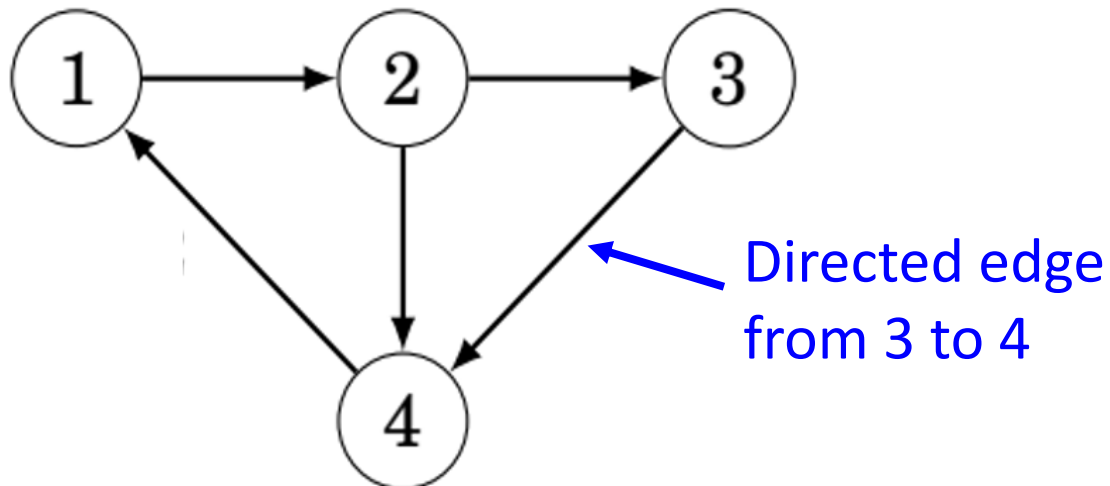
# การจำลองปัญหาด้วยกราฟ



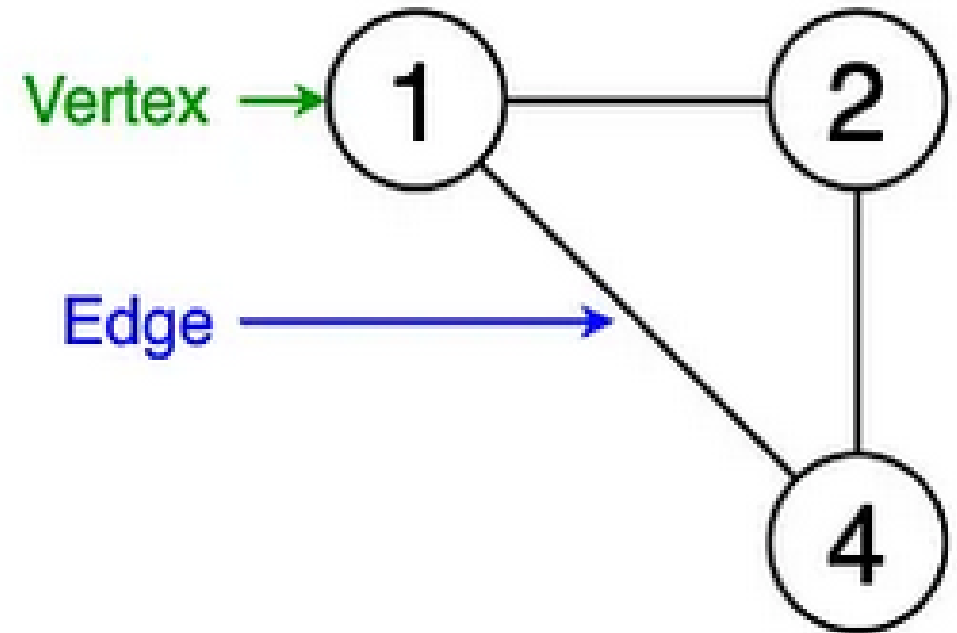
# เวอร์เท็กซ์ (vertex) และเอดจ์ (edge)

■ กราฟ  $G = (V, E)$  ถูกนิยามว่าเป็นเซตของเวอร์เท็กซ์ (vertex/vertices) และเอดจ์ (edge) โดยที่

- เวอร์เท็กซ์ ทำหน้าที่แทนเอนทิตี
- เอดจ์ คือความสัมพันธ์ระหว่างเวอร์เท็กซ์



กราฟทิศทาง

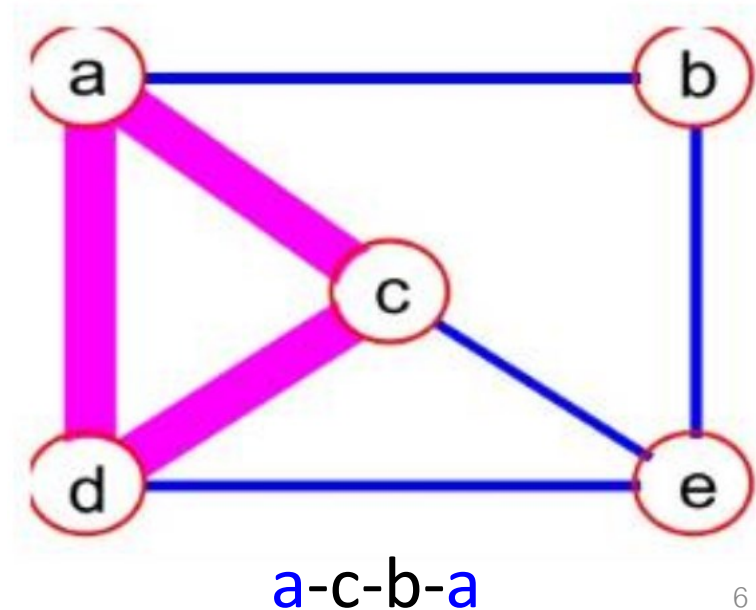
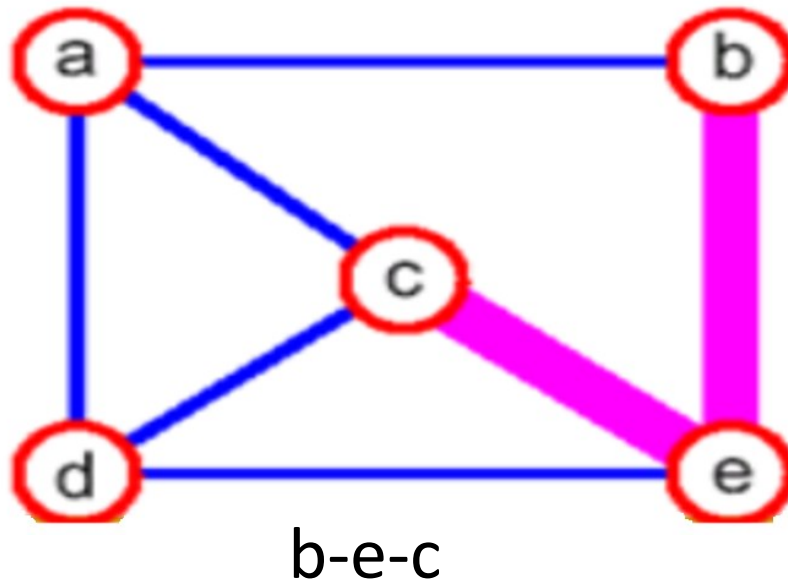
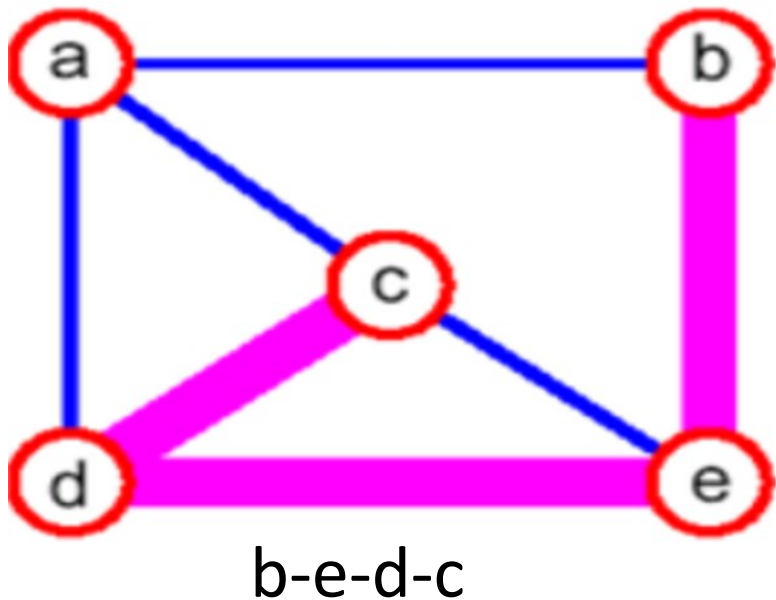


กราฟไม่มีทิศทาง



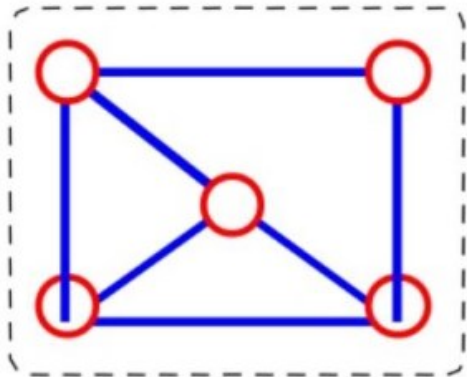
# เส้นทาง (path) และวงวน (cycle)

- Path คือลำดับ (Sequence) ของ vertices ที่เชื่อมด้วยเอดจ์เดียวกัน
  - ความยาวของ path คือจำนวนเอดจ์ที่เชื่อมระหว่างเวอร์เท็กซ์เริ่มต้นและเวอร์เท็กซ์สิ้นสุด
- Cycle คือ path ซึ่งลากผ่าน vertices โดยมีเวอร์เท็กซ์เริ่มต้นและเวอร์เท็กซ์สิ้นสุดเป็นเวอร์เท็กซ์เดียวกัน

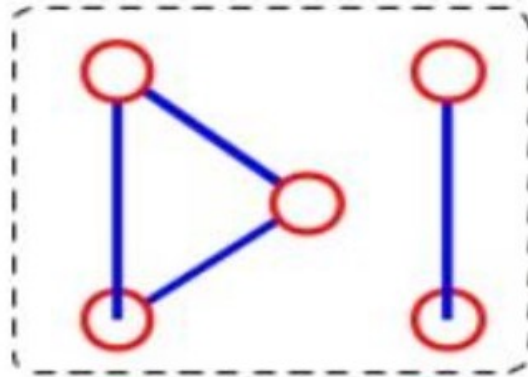


# Connected Component (subgraph)

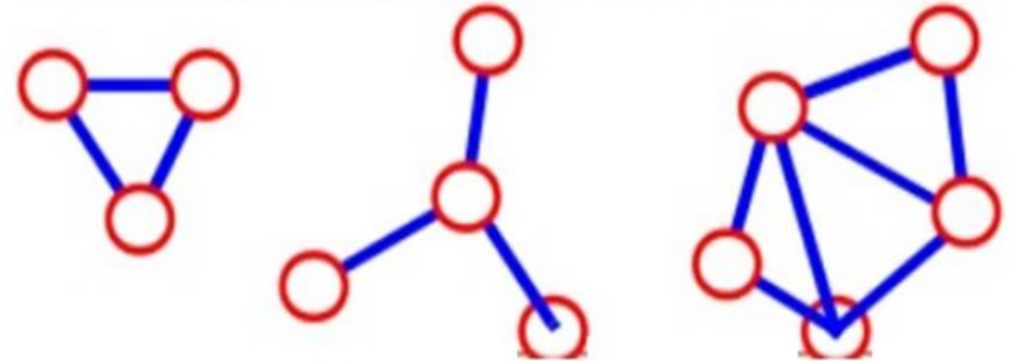
- connected graph คือกราฟที่มีเอดจ์เชื่อมโยงในทุกเวอร์เท็กซ์
- subgraph คือเซตย่อยของเวอร์เท็กซ์และเอดจ์ในกราฟ
- connected component คือ connected subgraphs ในกราฟ



connected



not connected



Connected components



# โครงสร้างข้อมูลกราฟ (Graph data structure)

- เมตริกซ์ประชิด (Adjacency Matrix)
  - มักใช้อาร์เรย์ 2 มิติ แทนเวอร์เท็กซ์ที่เชื่อมต่อกันด้วยเอดจ์
  - ข้อดีคือจัดการง่ายและทำงานได้รวดเร็ว
  - ข้อเสียคือสิ้นเปลืองหน่วยความจำ โดยเฉพาะอย่างยิ่ง sparse graph
- ลิสต์ประชิด (Adjacency List)
  - ใช้ลิสต์ร่วมกับอาร์เรย์ขนาด 1 มิติเพื่อเก็บเอดจ์ระหว่างเวอร์เท็กซ์
  - ข้อดีคือประหยัดหน่วยความจำที่ใช้ในการแสดงกราฟขนาดใหญ่
  - ข้อเสียคือยุ่งยากในการจัดการกว่าเมตริกซ์ประชิด



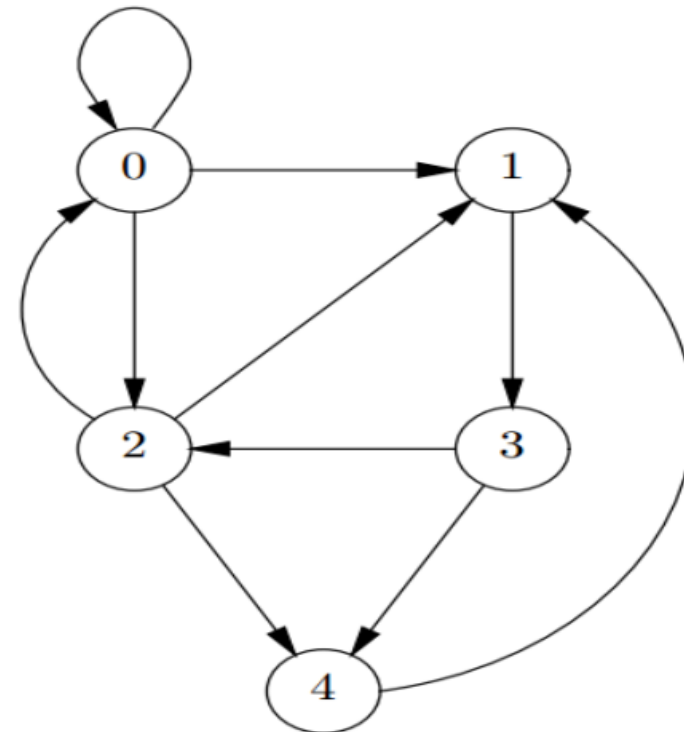
# เมตริกซ์ประชิด (adjacency matrix)

เวอร์เท็กซ์

เวอร์เท็กซ์

|   | 0 | 1 | 2 | 3 | 4 |
|---|---|---|---|---|---|
| 0 | 1 | 1 | 1 | 0 | 0 |
| 1 | 0 | 0 | 0 | 1 | 0 |
| 2 | 1 | 1 | 0 | 0 | 1 |
| 3 | 0 | 0 | 1 | 0 | 1 |
| 4 | 0 | 1 | 0 | 0 | 0 |

```
int edges[5][5];
```





# เมตริกซ์ประชิด (adjacency matrix)

```
#include <iostream>
#include <vector>

using namespace std;

int main() {
    int n_vertex = 5;

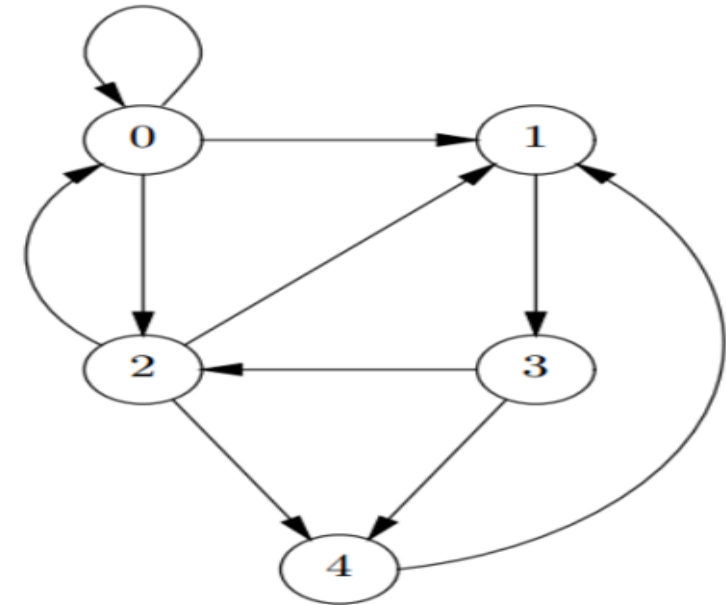
    // create 2D vector with n rows and n columns
    vector<vector<int>> graph(n_vertex, vector<int>(n_vertex, 0));

    int numVertices = graph.size(); // get no. rows

    graph[0][0] = 1; graph[0][1] = 1; graph[0][2] = 1;
    graph[1][3] = 1;
    graph[2][0] = 1; graph[2][1] = 1; graph[2][4] = 1;
    graph[3][2] = 1; graph[3][4] = 1;
    graph[4][1] = 1;

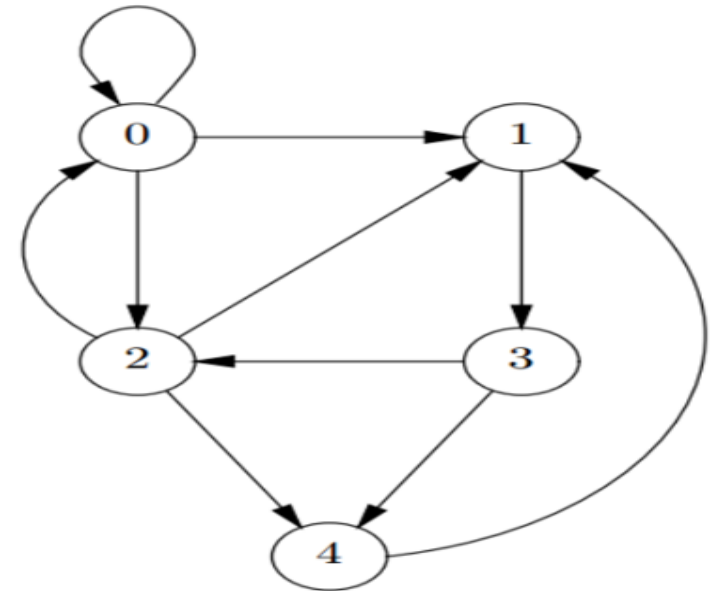
    print_graph(graph);

    return 0;
}
```



# เมตริกซ์ประชิด (Adjacency matrix)

```
print_graph(vector<vector<int>>& graph)
{ for(int i=0; i < graph.size(); i++)
  { cout << i << " : ";
    for(int j=0; j < graph[i].size(); j++)
      cout << " " << graph[i][j];
    cout<<endl;
  }
}
```

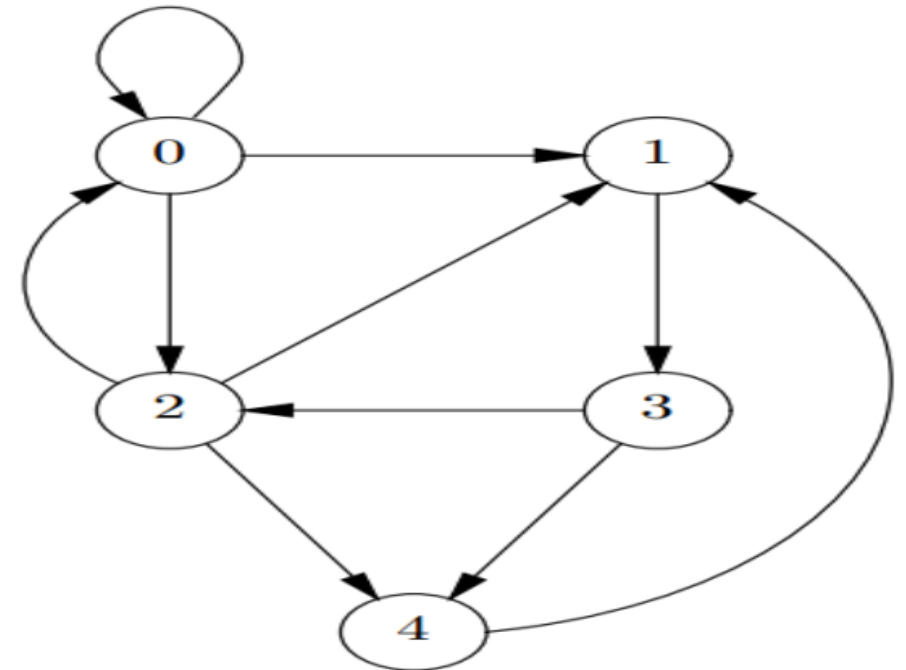
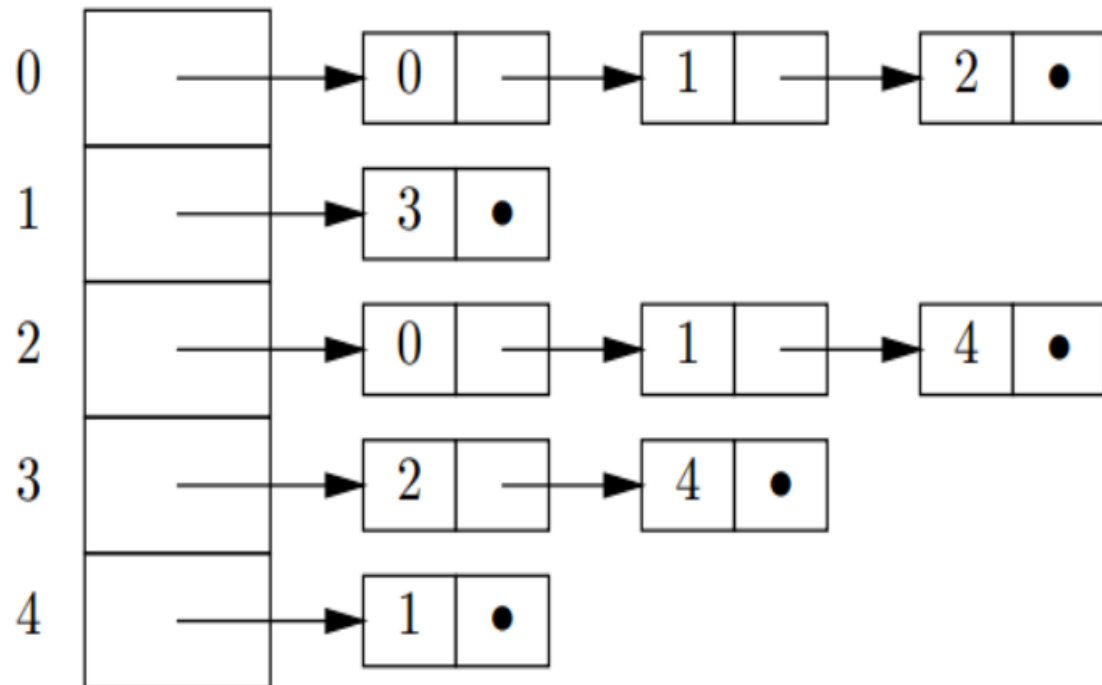


|   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|
| 0 | : | 1 | 1 | 1 | 0 | 0 |
| 1 | : | 0 | 0 | 0 | 1 | 0 |
| 2 | : | 1 | 1 | 0 | 0 | 1 |
| 3 | : | 0 | 0 | 1 | 0 | 1 |
| 4 | : | 0 | 1 | 0 | 0 | 0 |

# ลิสต์ประชิด (Adjacency list)

ประกอบไปด้วยอาร์เรย์ของลิสต์ของเอจด์ที่เชื่อมโยงในแต่ละเวอร์เท็กซ์ของกราฟ

successors





# ลิสต์ประชิดด้วยอาร์เรย์ของเวกเตอร์ (array of vector)

```
#include <iostream>
#include <vector>
```

```
using namespace std;
```

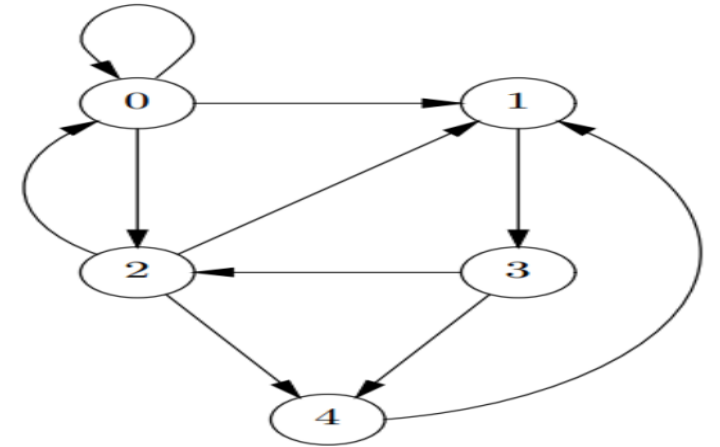
```
int main() {
    int n = 5;
    vector<int> adjList[n];

    // Adding edges to the graph
    adjList[0].push_back(0);
    adjList[0].push_back(2);
    adjList[2].push_back(0);
    adjList[2].push_back(4);
    adjList[3].push_back(4);

    printGraph(adjList, n);

    return 0;
}
```

```
adjList[0].push_back(1);
adjList[1].push_back(3);
adjList[2].push_back(1);
adjList[3].push_back(2);
adjList[4].push_back(1);
```

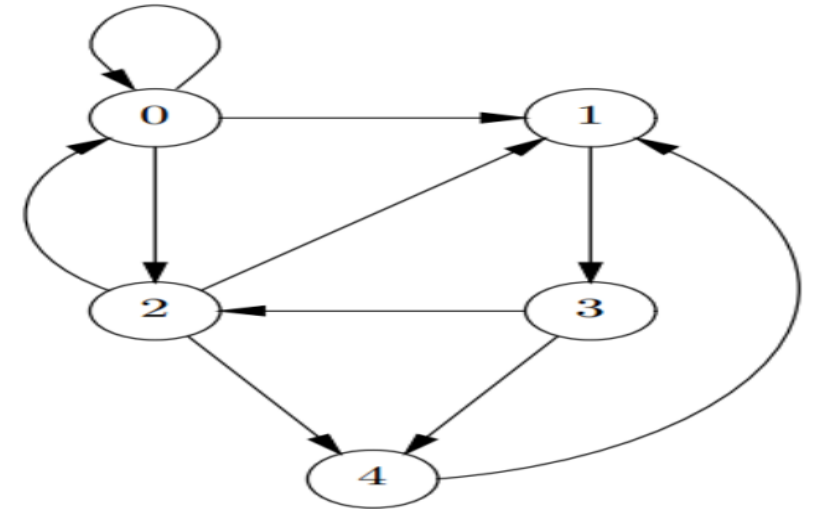




# ลิสต์ประชิดด้วยอาร์เรย์ของเวกเตอร์ (array of vector)

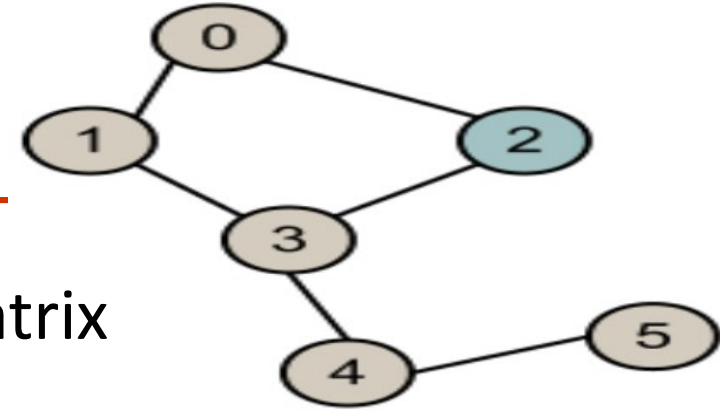
```

void printGraph(vector<int> adjList[], int n) {
    for (int i = 0; i < n; ++i) {
        cout << i << ": ";
        for (int j=0; j<adjList[i].size(); j++) {
            cout << adjList[i][j] << " ";
        }
        cout << endl;
    }
}
    
```



```

0: 0 1 2
1: 3
2: 0 1 4
3: 2 4
4: 1
    
```



1. จงเขียนโปรแกรมเพื่อจัดเก็บกราฟด้านขวามือด้วยวิธี **adjacency matrix** พร้อมแสดงผลลัพธ์ในรูปแบบ **adj matrix**

2. จงเขียนโปรแกรมเพื่ออ่านข้อมูลเพื่อสร้างกราฟทิศทางด้วยวิธี **adjacency list** พร้อมแสดงผลลัพธ์ในรูปแบบ **adj. list** โดยที่

- บรรทัดแรก หมายถึง จำนวนเวอร์เท็กซ์ **n**
- บรรทัดสอง หมายถึง จำนวนเอดจ์ **m**
- **m** บรรทัดที่เหลือเป็นรายการเอดจ์ในกราฟ

input

5

6

1 2

2 3

3 4

4 5

4 6

5 2

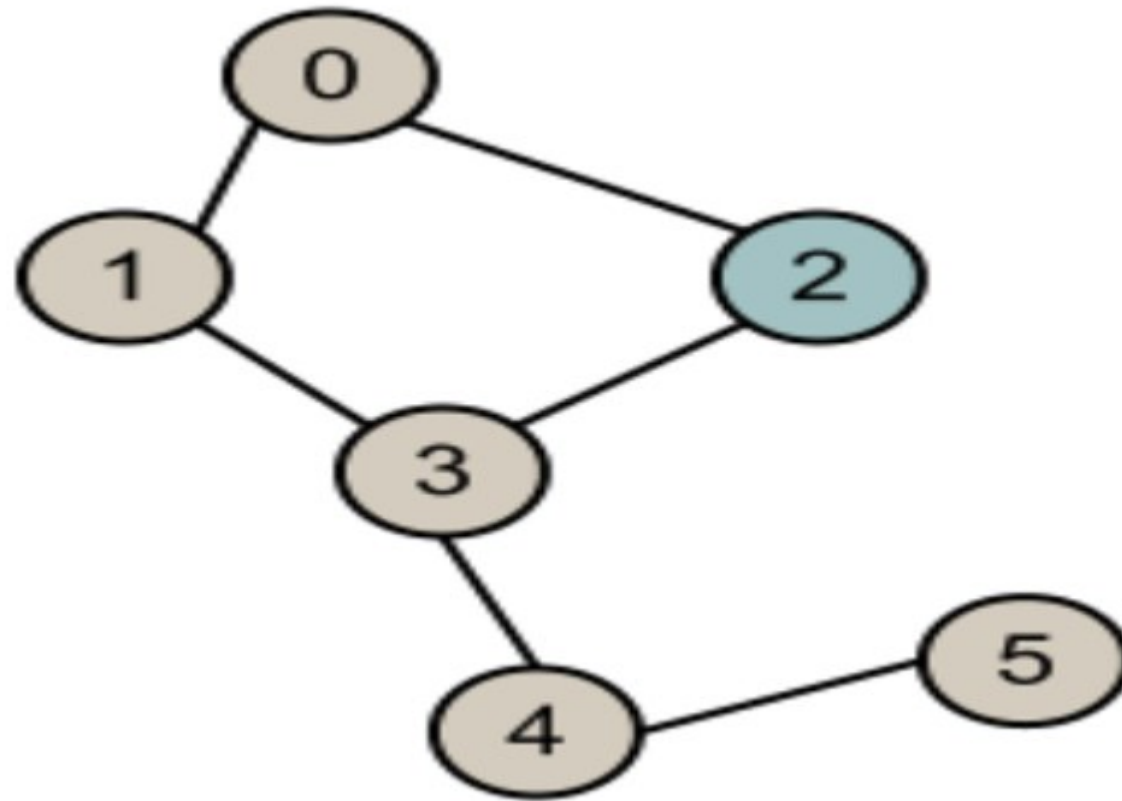
output

```
1: 2
2: 3
3: 4
4: 5 6
5: 2
```

# การสำรวจกราฟ (Graph Exploration)

- ปัญหาพื้นฐานที่สำคัญที่สุดของกราฟ คือการสำรวจกราฟ (graph exploration)
- สองอัลกอริทึมที่นิยมใช้ในการสำรวจกราฟ
  - Breadth First Search (BFS)
  - Depth First Search (DFS)

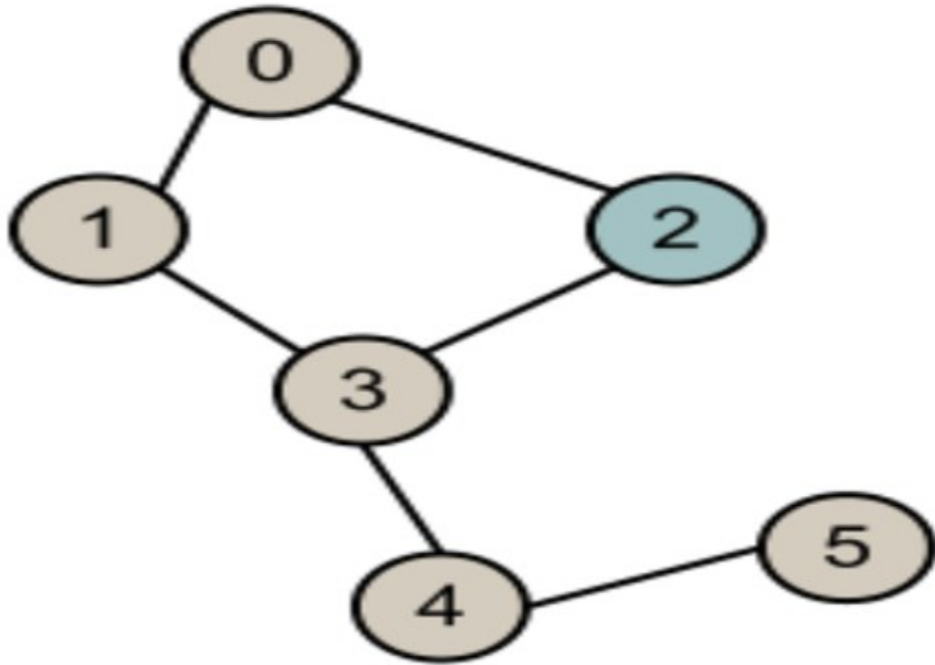
# การสำรวจกราฟด้วย BFS



# Breadth First Search (BFS)

1. กำหนดเวอร์เท็กซ์เริ่มต้น (source vertex) เป็น current vertex
2. สืบหา neighbors ของ current vertex ซึ่งยังไม่เคยถูก visit โดยนำไปเก็บไว้ในคิว (Queue)
3. ดึง neighbors จากคิว ทำซ้ำในขั้นตอน 2
4. จบการทำงานเมื่อทุกเวอร์เท็กซ์ถูก visit หรือไม่มีเวอร์เท็กซ์ใดอยู่ในคิว

# Breadth First Search



1.  $Q = \{ \}$

2.  $Q = \{ 2 \}$     เพิ่ม vertex 2 ลงในคิว

ดึง 2 ออกจากคิว    ตรวจสอบเอดจ์ neighbor ของ 2 ได้แก่ 0, 3

3.  $Q = \{ 0, 3 \}$     เพิ่ม vertex 0, 3 ลงในคิว

ดึง 0 ออกจากคิว    ตรวจสอบเอดจ์ neighbor ของ 0 ได้แก่ 1, 3 แต่  
เนื่องจาก 3 อยู่ในสถานะรอการสำรวจ จึงเพิ่มเฉพาะ 1

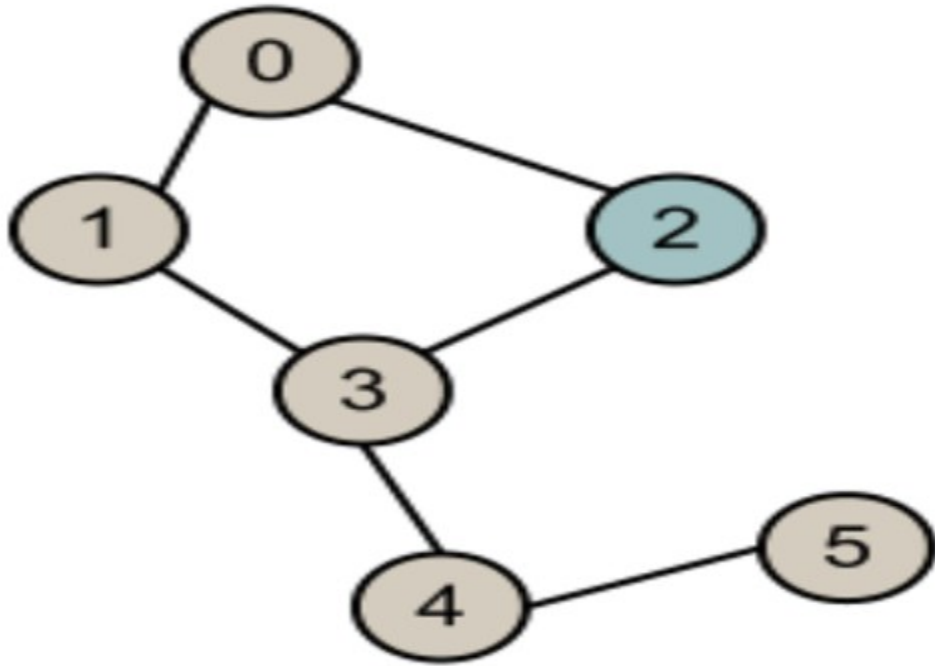
4.  $Q = \{ 3, 1 \}$     เพิ่ม vertex 1 ลงในคิว

ดึง 3 ออกจากคิว    ตรวจสอบ neighbor ของ 3 คือ 4

ลำดับในการ visit Q คือ **2 0 3 1 4 5**



# Breadth First Search



5.  $Q = \{1, 4\}$     เพิ่ม 4 ลงในคิว

ดึง 1 ออกจากคิว    ตรวจสอบเอดจ์ neighbor ของ 1

6.  $Q = \{4\}$

ดึง 4 ออกจากคิว    ตรวจสอบเอดจ์ neighbor ของ 4 คือ 5

7.  $Q = \{5\}$     เพิ่ม 5 ลงในคิว

ดึง 5 ออกจากคิว    ตรวจสอบ neighbor 5

8.  $Q = \{ \}$     จบการทำงานเมื่อคิวว่าง

ลำดับในการ visit Q คือ **2 0 3 1 4 5**

# Breadth First Search (BFS)

```
#include <iostream>
#include <vector>
#include <queue>

using namespace std;
void BFS(vector<vector<int>> graph, int startVertex, int V) {
    vector<bool> visited(V, false);
    queue<int> queue;
    queue.push(startVertex);
    visited[startVertex] = true;

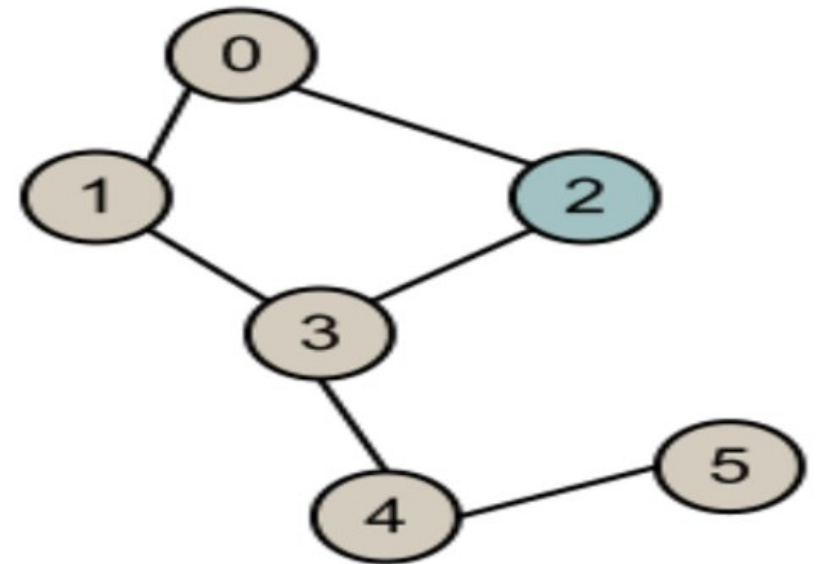
    while (!queue.empty()) {
        int currentVertex = queue.front();
        queue.pop();
        cout << currentVertex << " ";

        for (int i = 0; i < graph.size(); ++i) {
            if (graph[currentVertex][i] && !visited[i]) {
                queue.push(i);
                visited[i] = true;
            }
        }
    }
}
```

# Breadth First Search (BFS)

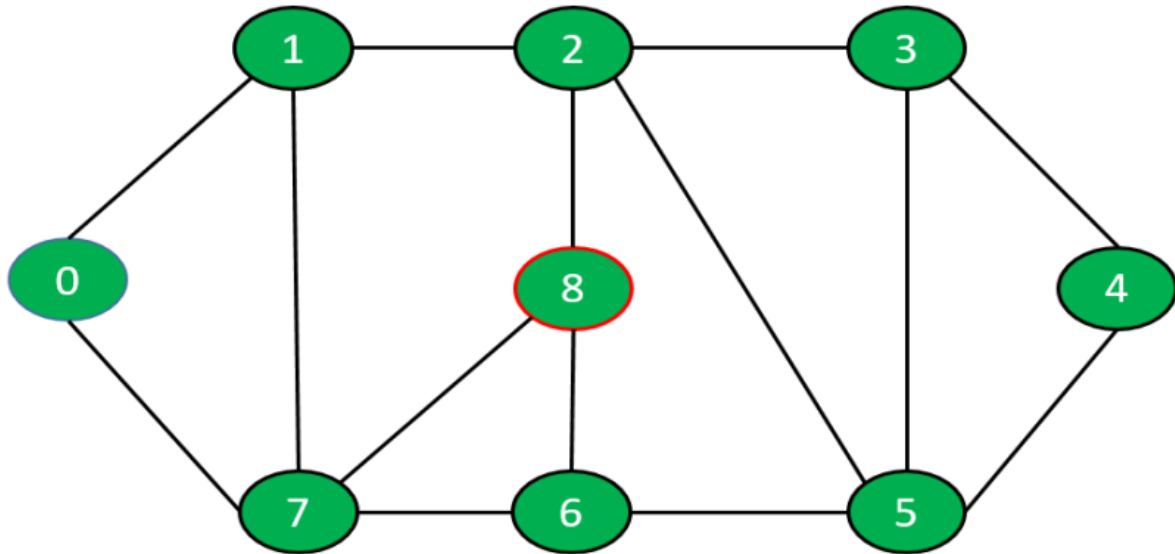
```
int main() {  
    vector<vector<int>> graph = {  
        {0, 1, 1, 0, 0, 0}, {1, 0, 0, 1, 0, 0},  
        {1, 0, 0, 1, 0, 0}, {0, 1, 1, 0, 1, 0},  
        {0, 0, 0, 1, 0, 1}, {0, 0, 0, 0, 1, 0}  
    };  
    int numVertices = graph.size();  
  
    // Start BFS from vertex 2  
    BFS(graph, 2, numVertices);  
  
    return 0;  
}
```

2 0 3 1 4 5



# แบบฝึกหัด

จงแสดงลำดับของเวอร์เท็กซ์ที่ถูกสำรวจโดย BFS กำหนดให้เวอร์เท็กซ์เริ่มต้นคือ 8

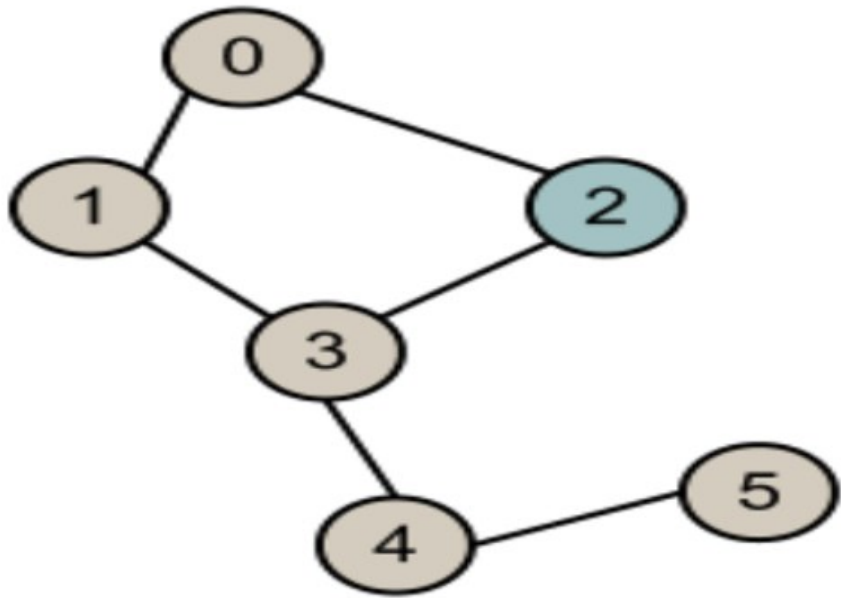


| ลำดับที่ | Current Vertex | Queue |
|----------|----------------|-------|
| 0        | -              | 8     |
| 1        | 8              | 2 7 6 |
| 2        |                |       |
| 3        |                |       |
| 4        |                |       |
| 5        |                |       |
| 6        |                |       |
| 7        |                |       |
| 8        |                |       |
| 9        |                |       |

# Depth First Search (DFS)

- ขั้นตอนที่ 1 กำหนด start vertex เป็น current vertex
- ขั้นตอนที่ 2 เช็คนะighbor ของ current vertex ว่าถูกสำรวจหรือไม่
- ขั้นตอนที่ 3 หาก neighbor ในขั้นตอน 2 ยังไม่เคยถูกสำรวจ จะทำการเก็บ current vertex ลงใน stack และเปลี่ยนสถานะของ neighbor ให้เป็น current vertex
- ขั้นตอนที่ 4 หากไม่มี neighbor ที่สำรวจไปได้ จะดึง vertex ใน stack ออกมาและเปลี่ยนสถานะให้กลายเป็น current vertex
- ขั้นตอนที่ 5 ทำซ้ำขั้นตอนที่ 2 จน stack ว่าง

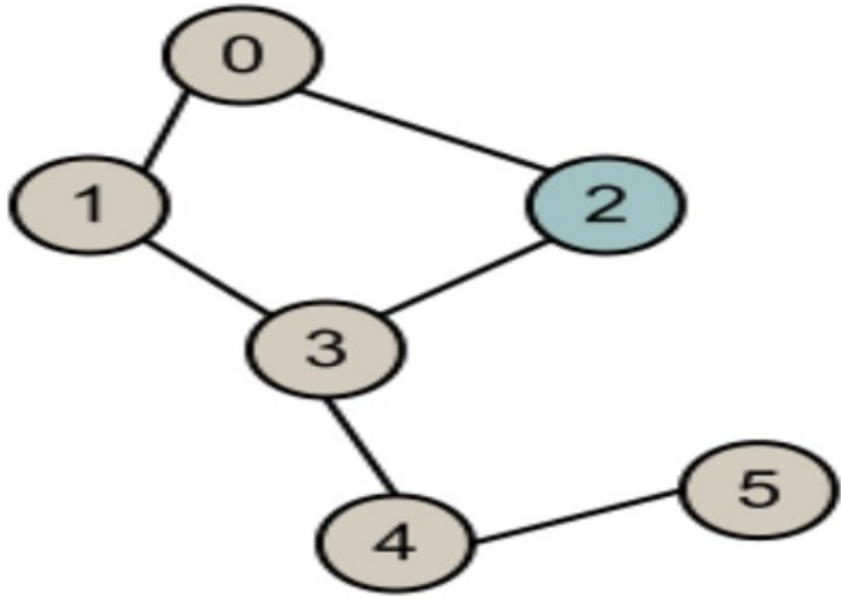
# Depth First Search (DFS)



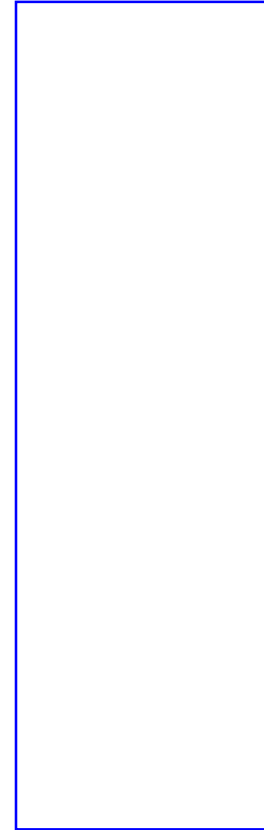
PUSH 2

2

# Depth First Search (DFS)



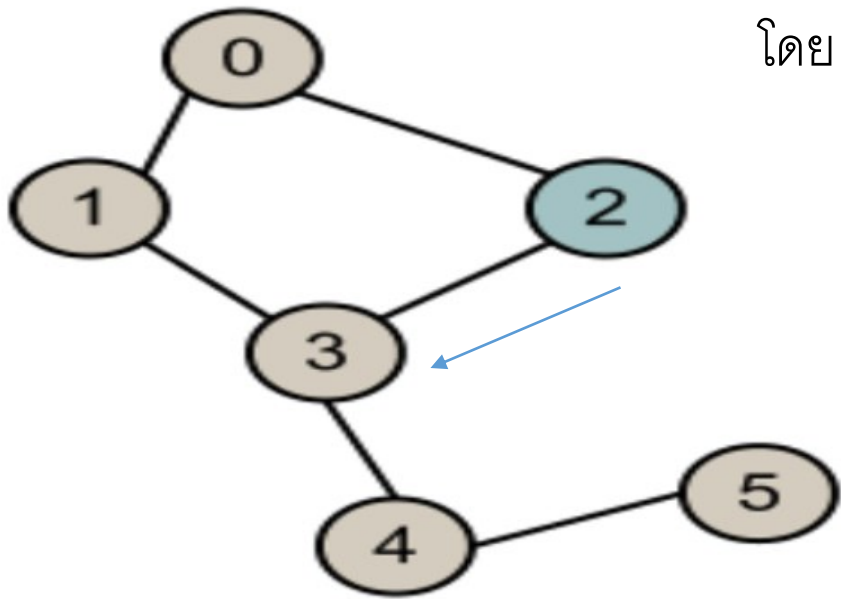
POP 2



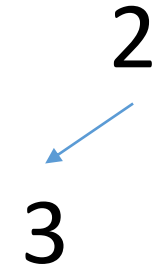
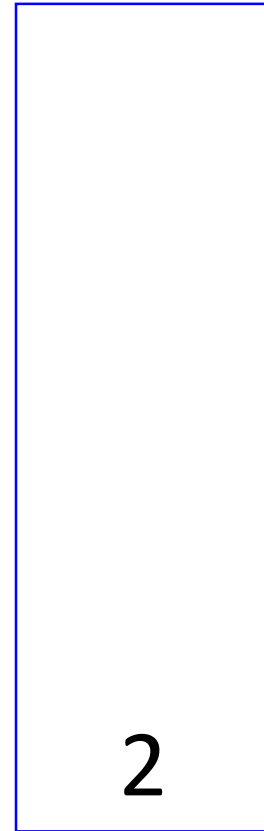
2



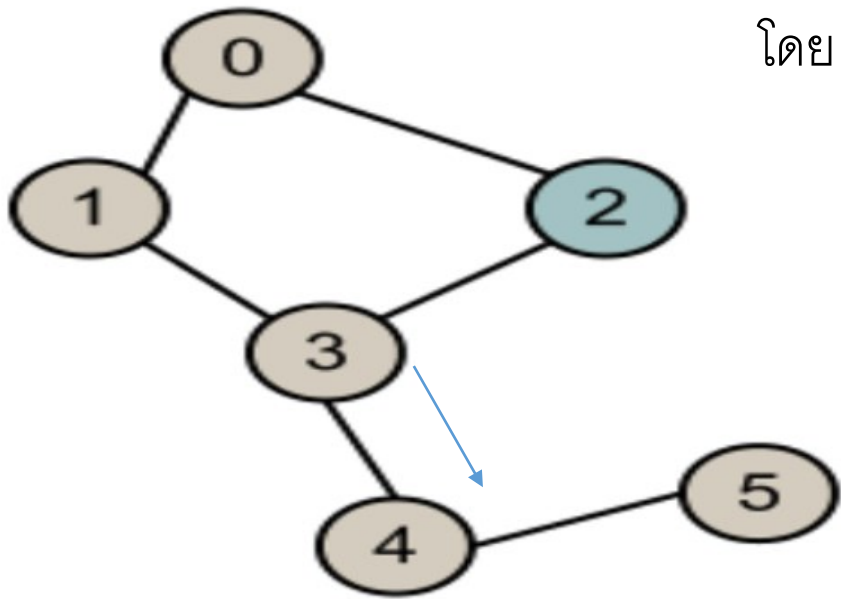
# Depth First Search (DFS)



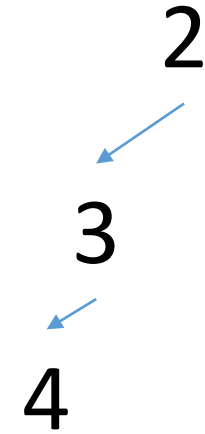
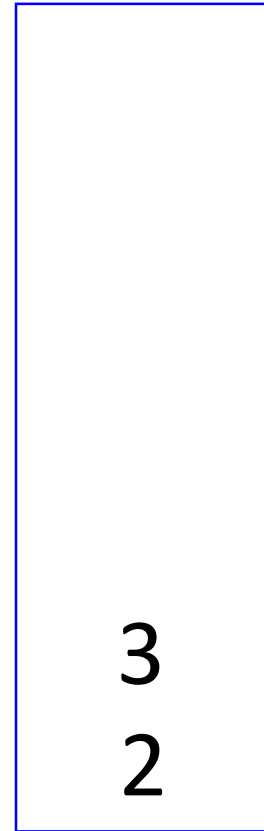
สำรวจ neighbor 3  
โดย PUSH 2



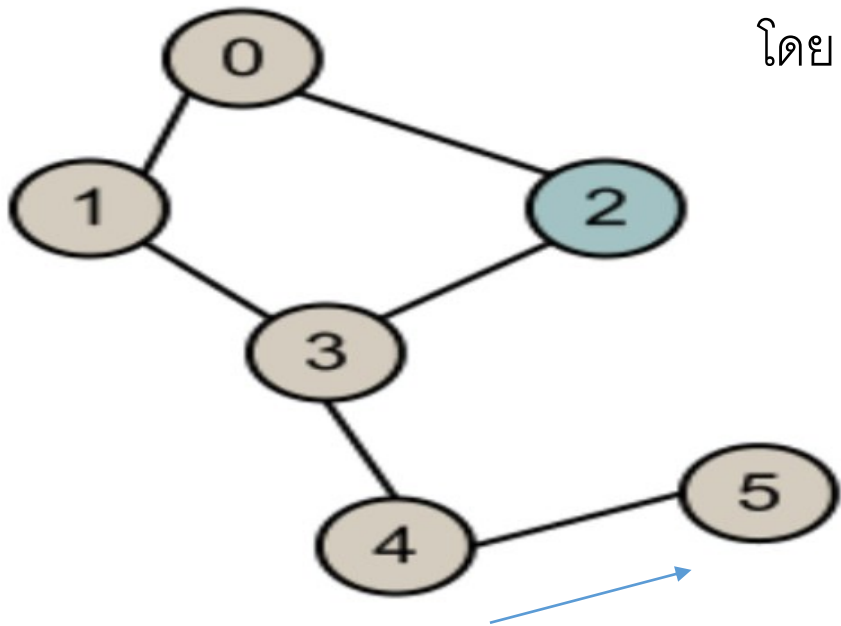
# Depth First Search (DFS)



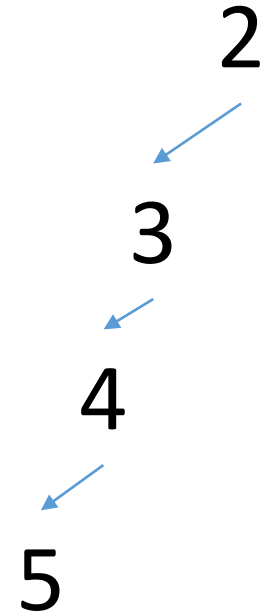
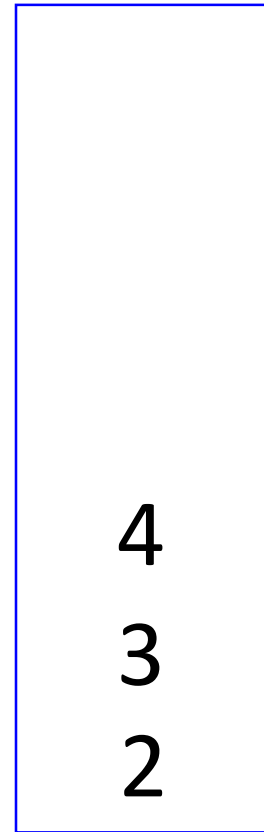
สำรวจ neighbor 4  
โดย PUSH 3



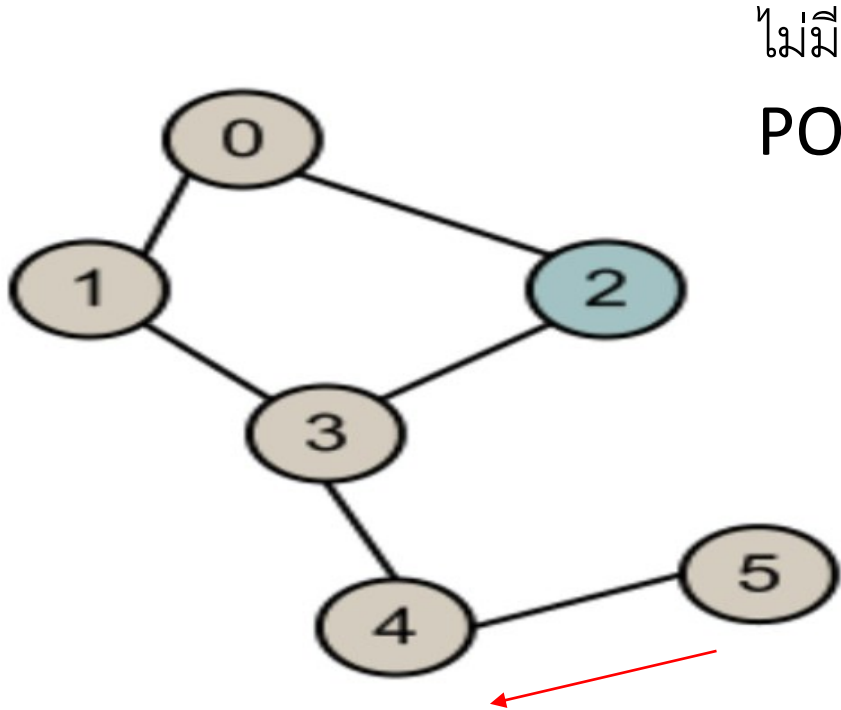
# Depth First Search (DFS)



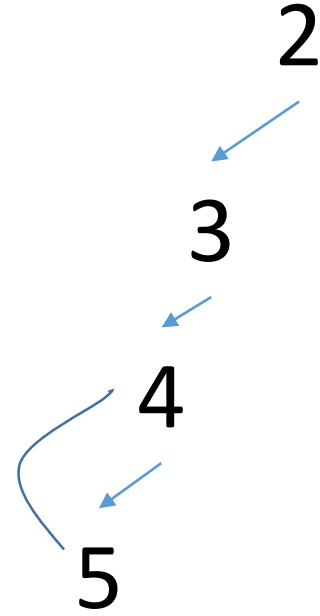
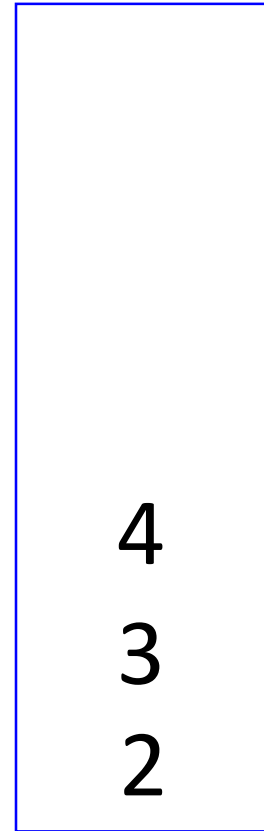
สำรวจ neighbor 5  
โดย PUSH 4



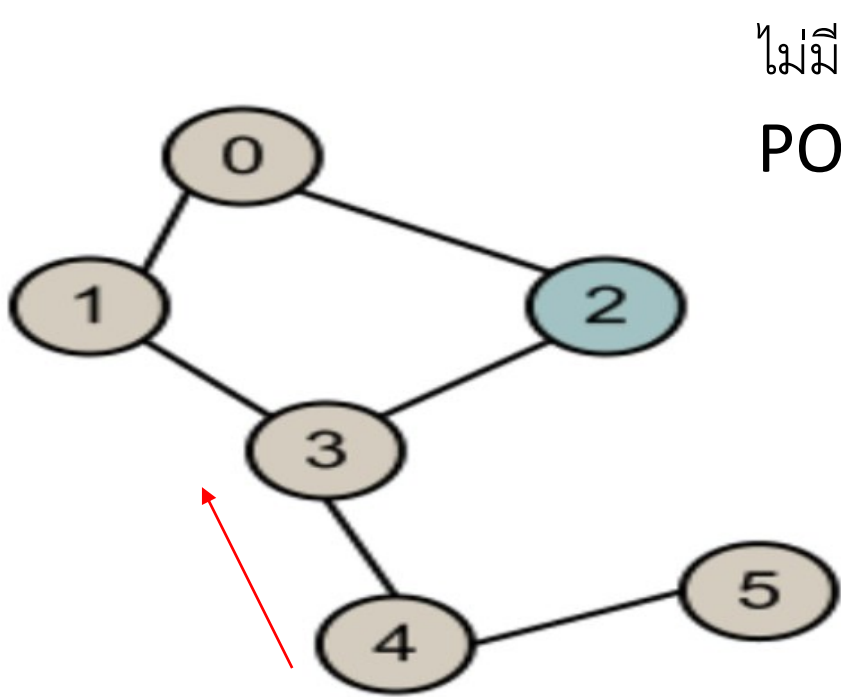
# Depth First Search (DFS)



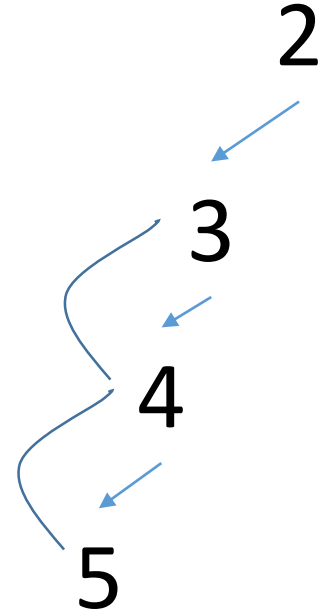
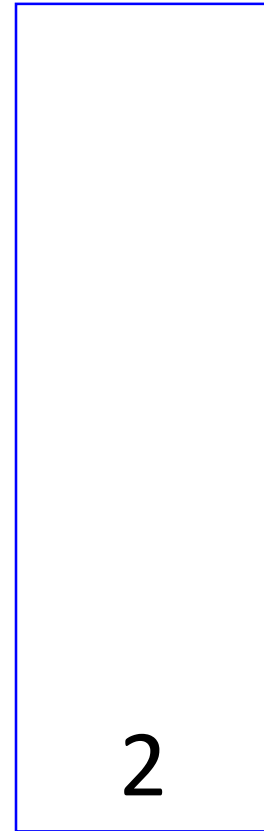
visited neighbor  
POP 4



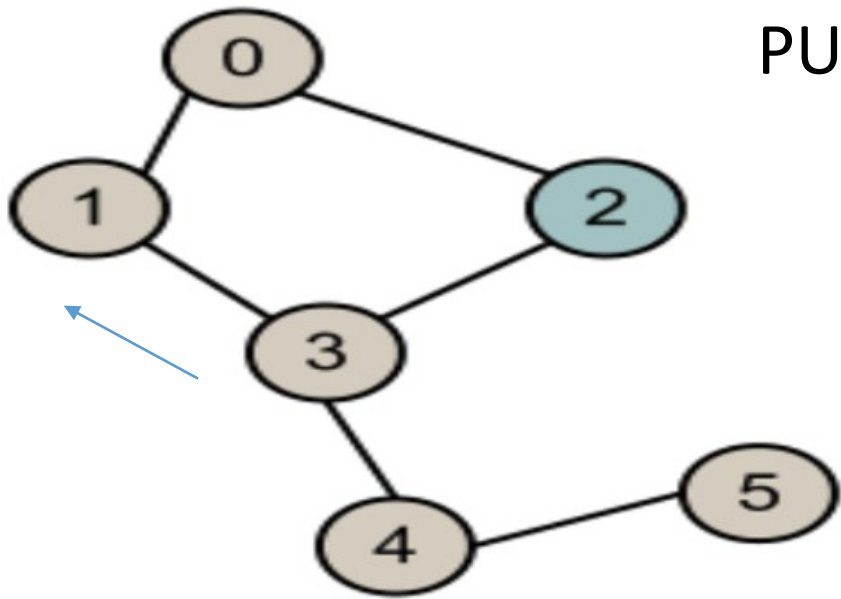
# Depth First Search (DFS)



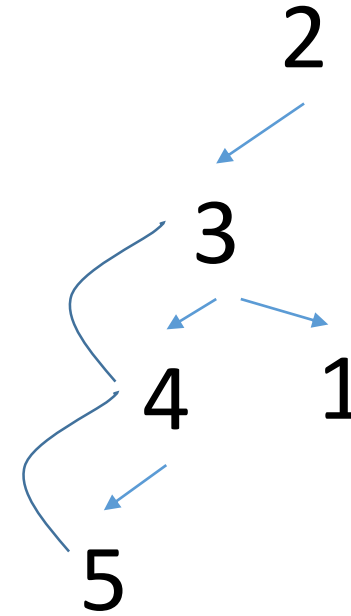
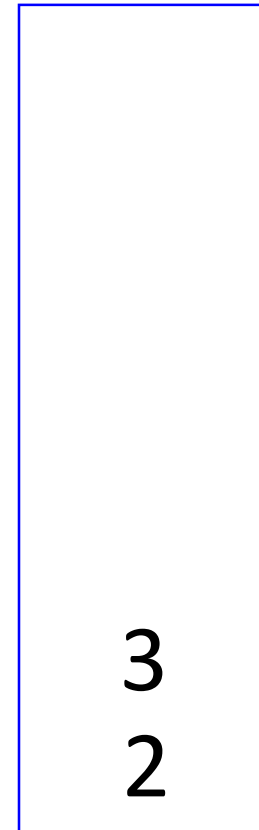
ไม่มี neighbor  
POP 3



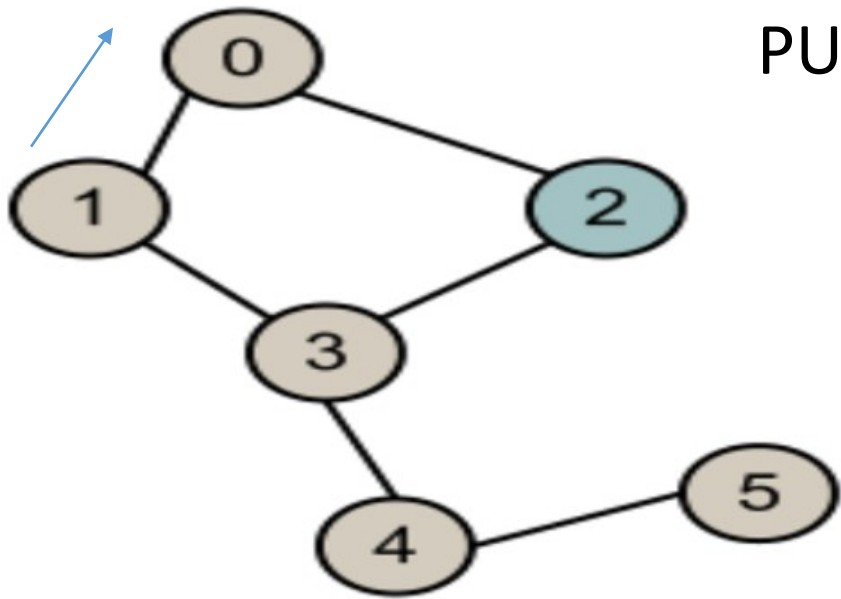
# Depth First Search (DFS)



สำรวจ neighbor 1  
PUSH 3

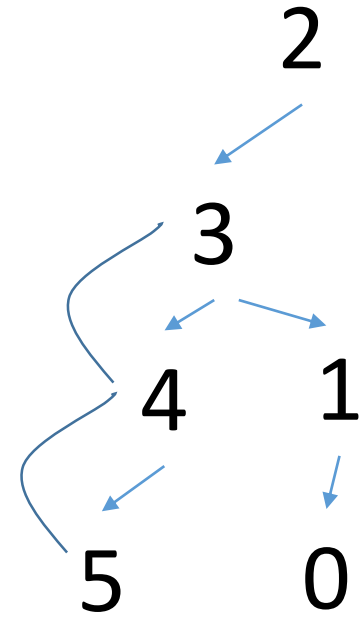


# Depth First Search (DFS)

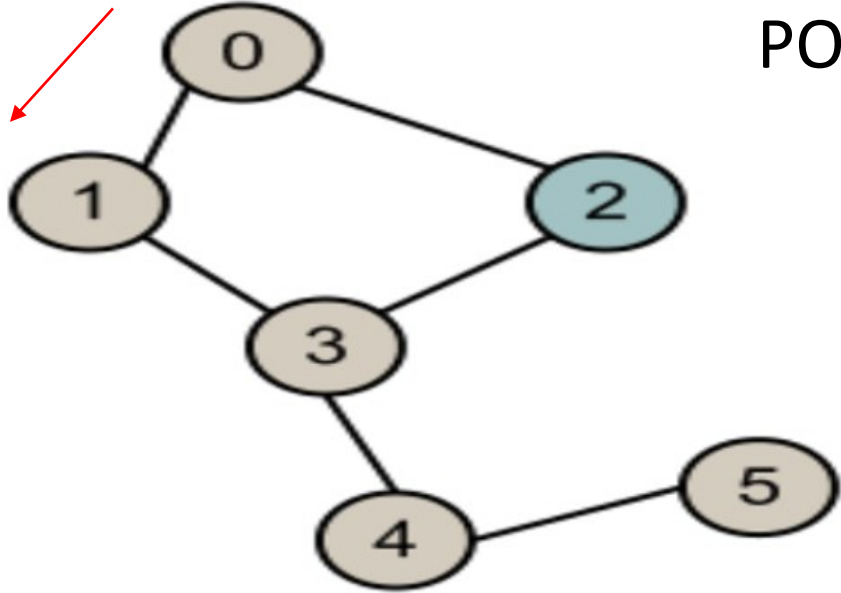


สำรวจ neighbor 0  
PUSH 1

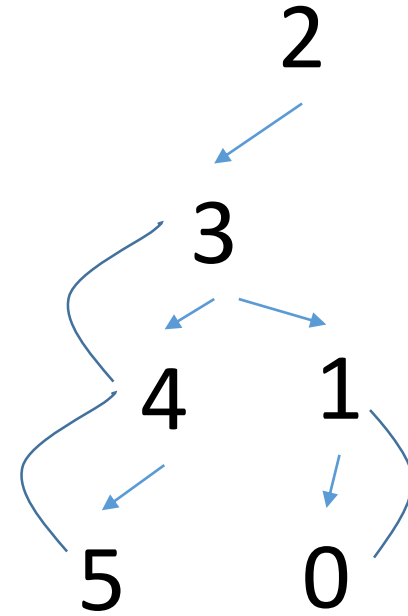
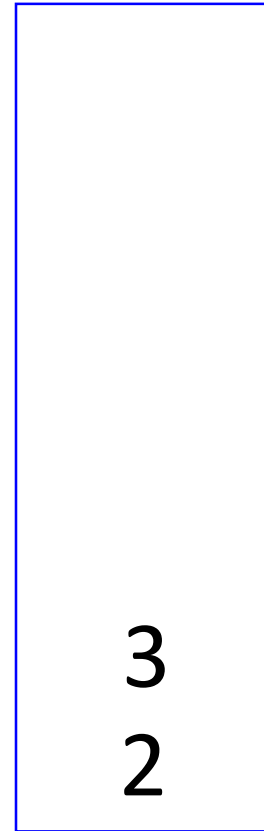
1  
3  
2



# Depth First Search (DFS)

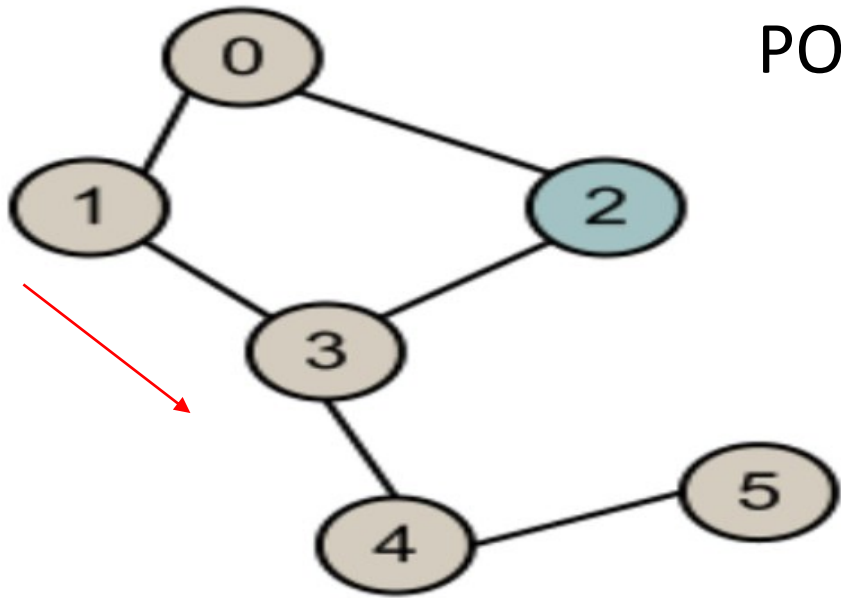


ไม่มี neighbor  
POP 1

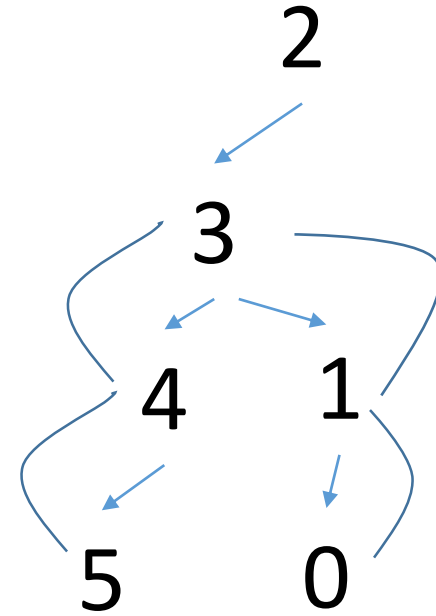
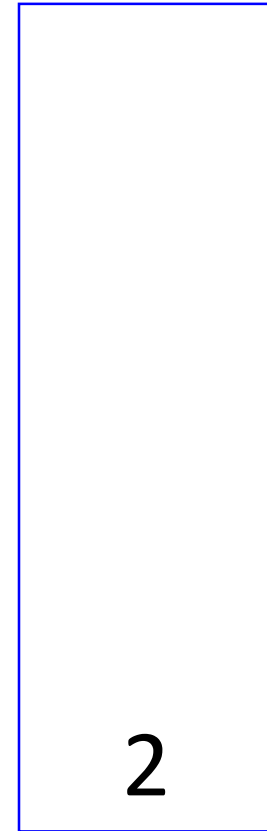




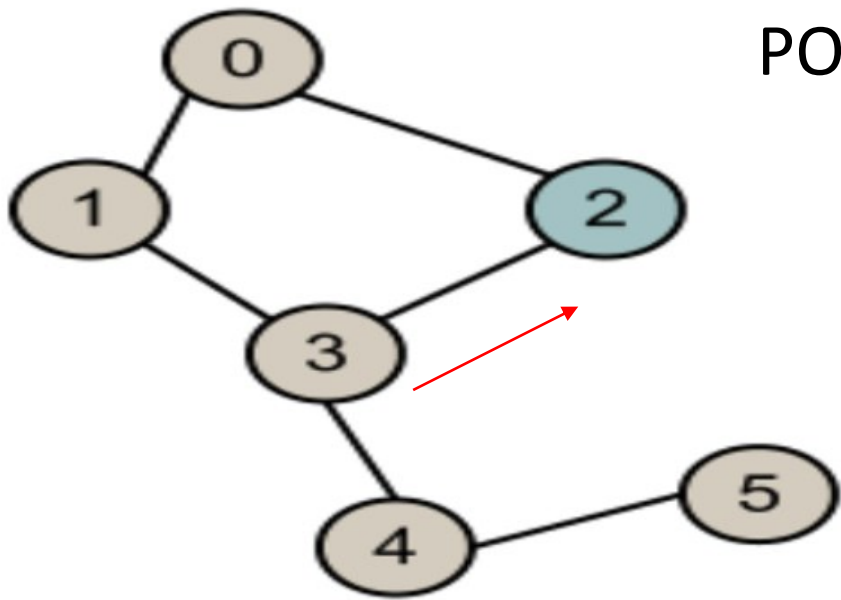
# Depth First Search (DFS)



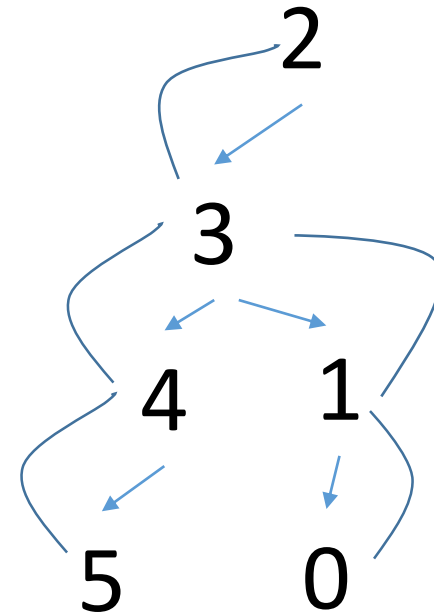
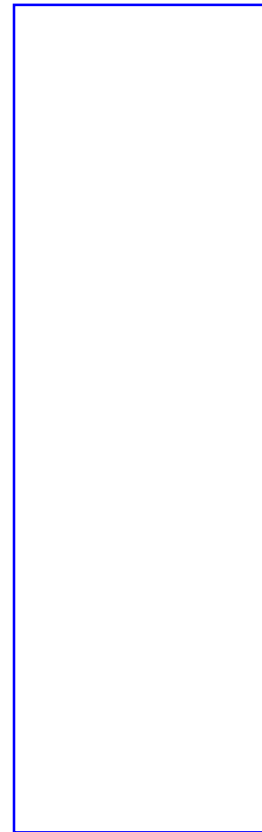
visited neighbor  
POP 3



# Depth First Search (DFS)



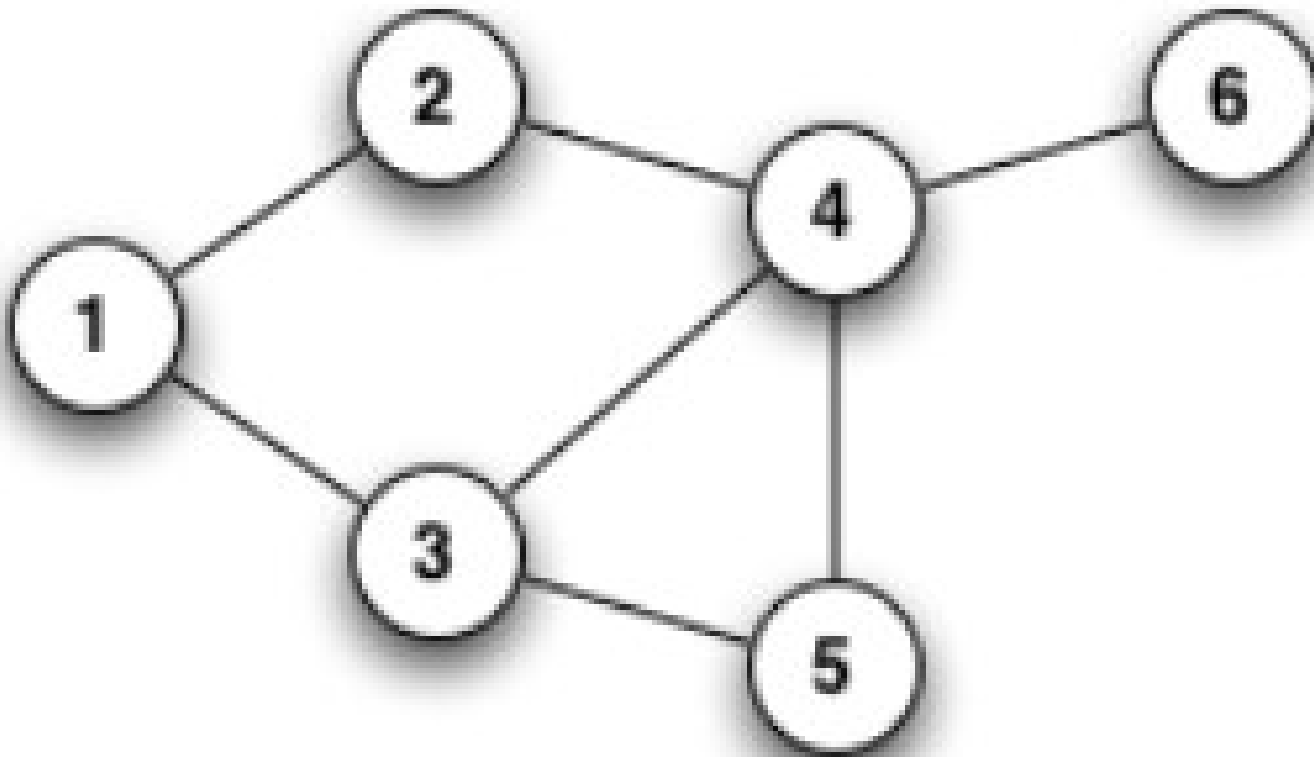
ไม่มี neighbor  
POP 2



ลำดับในการเยี่ยมชมเวอร์เทกซ์ 2 3 4 5 1 0

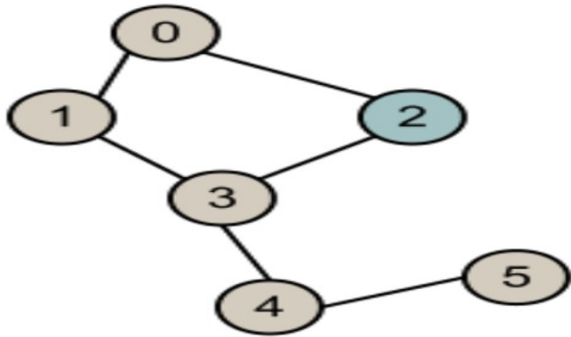
# แบบฝึกหัด

จงแสดงลำดับของเวอร์เท็กซ์ที่ถูกสำรวจโดย DFS กำหนดให้เวอร์เท็กซ์เริ่มต้นคือ 1



# Depth First Search (DFS)

```
void DFS (vector<vector<int>> graph, int startVertex, vector<bool>& visited) {  
  
    cout<<startVertex<< " ";  
    visited[startVertex] = true;  
  
    for (int i = 0; i < graph.size(); i++) {  
        if (graph[startVertex][i] & !visited[i]) {  
            DFS(graph, i, visited);  
        }  
    }  
}
```



2 0 1 3 4 5

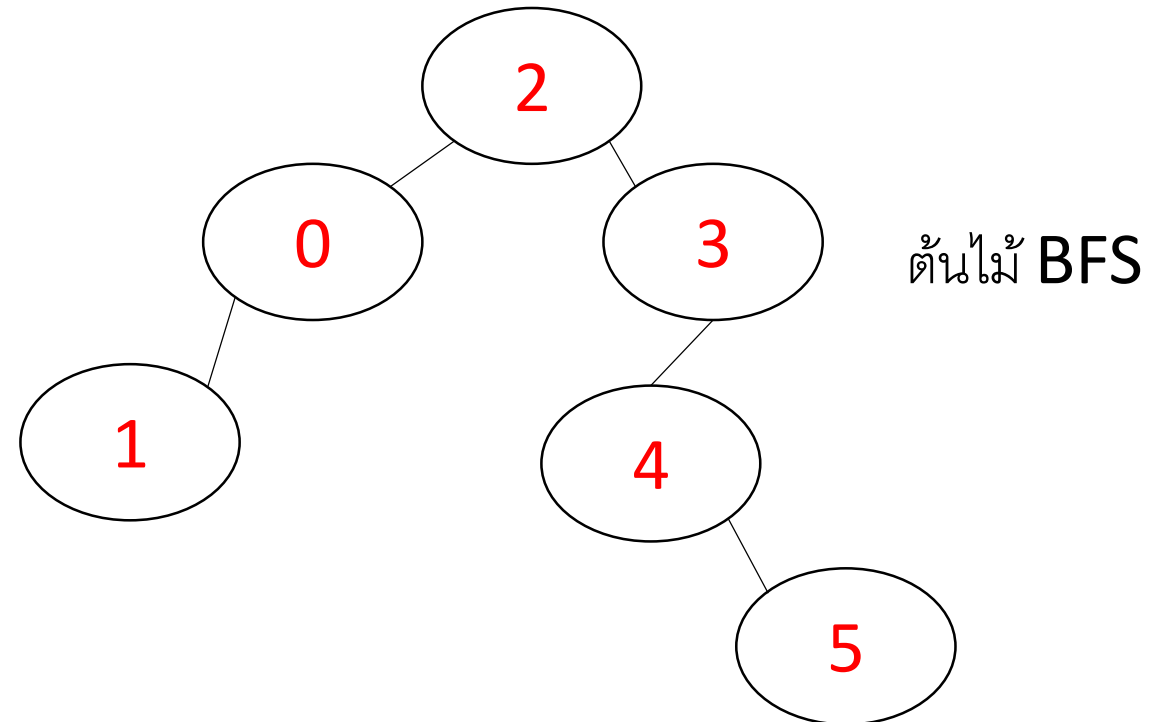
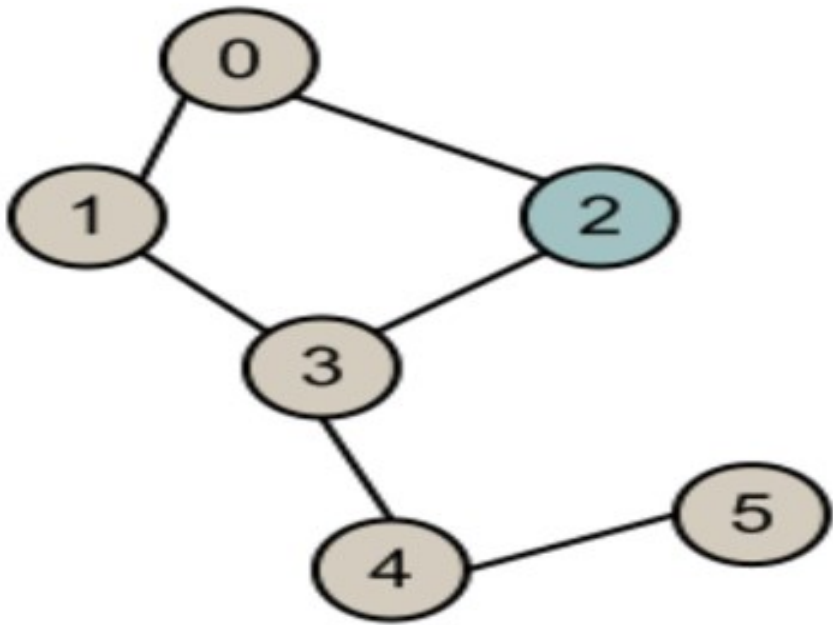
```
int main() {  
    vector<vector<int>> graph = {  
        {0, 1, 1, 0, 0, 0}, {1, 0, 0, 1, 0, 0},  
        {1, 0, 0, 1, 0, 0}, {0, 1, 1, 0, 1, 0},  
        {0, 0, 0, 1, 0, 1}, {0, 0, 0, 0, 1, 0}  
    };  
  
    int V = 6;  
    int startVertex = 2;  
  
    vector<bool> visited(V, false);  
    DFS(graph, startVertex, visited);  
  
    return 0;  
}
```

# แอปพลิเคชันของ BFS & DFS

- ค้นหา Shortest Path ในกราฟ
- ค้นหาวงวนในกราฟทิศทาง (Detect Cycle in a directed graph)
- ค้นหา Connected components (Subgraphs)
- การตรวจสอบความต่อเนื่องในกราฟ (Graph Connectivity)

# ค้นหา Shortest Path ในกราฟด้วย BFS

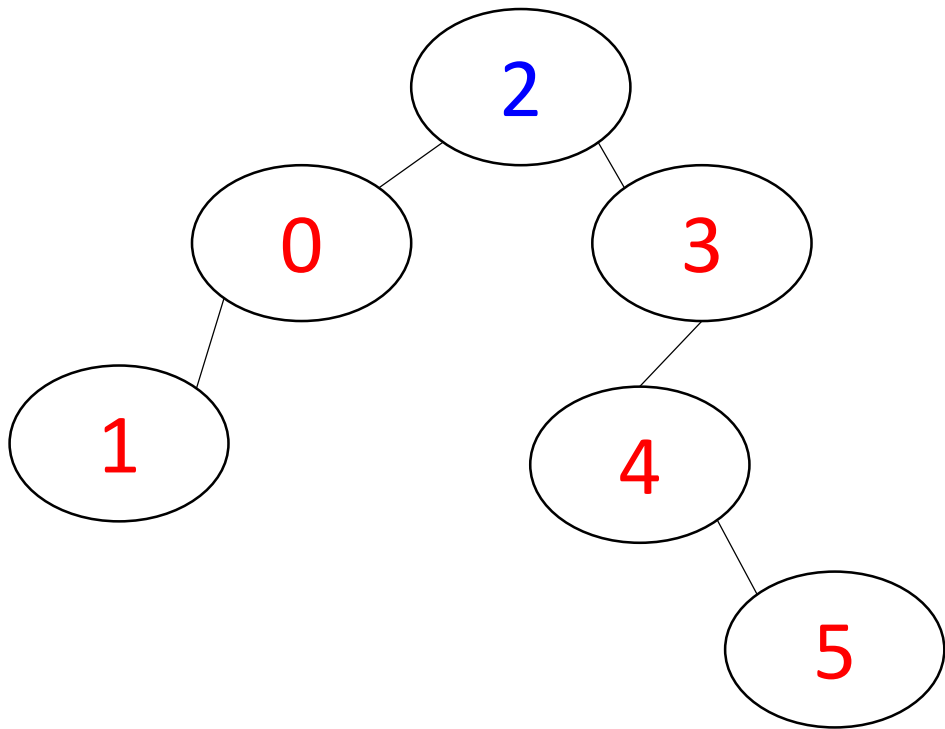
- ลำดับของเวอร์เท็กซ์ซึ่งถูกสำรวจโดย BFS สามารถแสดงในรูปของต้นไม้ความสัมพันธ์ระหว่าง neighbors เรียกว่า ต้นไม้ BFS
- ต้นไม้ BFS สามารถใช้ค้นหา shortest path จาก start vertex ได้



Shortest path:  $2 \rightarrow 0 = 1$ ,  $2 \rightarrow 1 = 2$ ,  $2 \rightarrow 4 = 2$ ,  $2 \rightarrow 5 = 3$

# Parent-child array

- อาร์เรย์ parent-child ใช้บันทึกต้นไม้ BFS โดยจะบันทึกค่า current vertex ทุกครั้งที่มีการสำรวจเอดจ์ neighbor ของมัน



| neighbor      | 0 | 1 | 2  | 3 | 4 | 5 |
|---------------|---|---|----|---|---|---|
| Parent vertex | 2 | 0 | -1 | 2 | 3 | 4 |

เก็บสถานะว่าเป็น  
start vertex

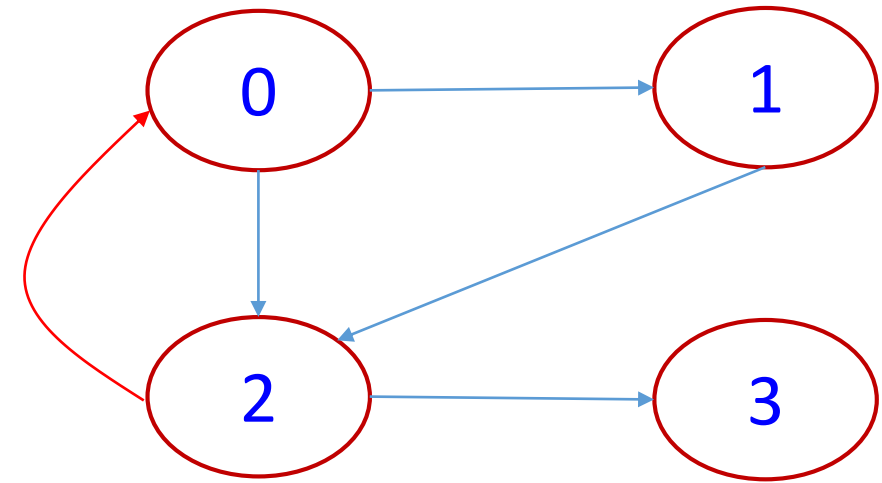
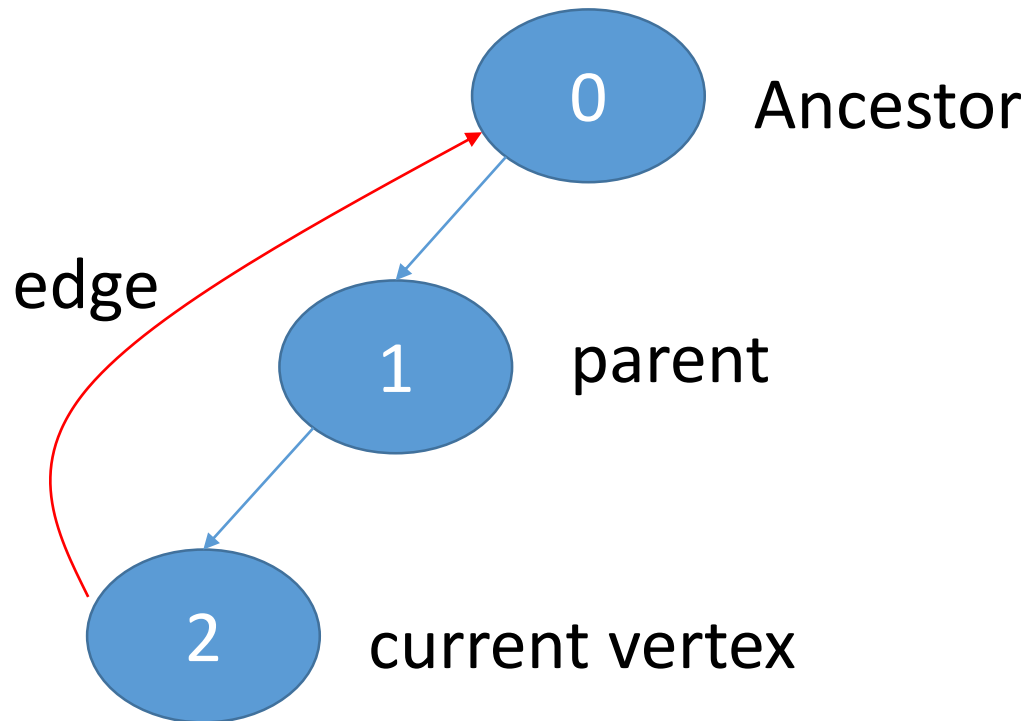
# parent-child array ใน BFS

```
void BFS(vector<vector<int>> graph, int startVertex, int V) {  
    vector<bool> visited(V, false);  
    vector<int> parent(V, -1);    // parent-child array  
  
    queue<int> queue;  
    queue.push(startVertex);  
    visited[startVertex] = true;  
  
    while (!queue.empty()) {  
        int currentVertex = queue.front();  
        queue.pop();  
        cout << currentVertex << " ";  
  
        for (int i = 0; i < graph.size(); ++i) {  
            if (graph[currentVertex][i] && !visited[i]) {  
                queue.push(i);  
                visited[i] = true;  
                parent[i] = currentVertex;    //update parent vertex  
            }  
        }  
    }  
}
```



# การตรวจนับวงวนในกราฟทิศทาง

- ระหว่างการสำรวจกราฟด้วย DFS หากกราฟมีวงวน (cycle) จะปรากฏ **back edge**
- back edge คือเอดจ์ที่เชื่อมระหว่าง current vertex และ ancestor ของมัน
- ดังนั้นการตรวจนับ cycle จะสามารถทดสอบได้จาก back edge ของ current vertex ซึ่งไม่ใช่ parent ของมัน



Directed graph

# การตรวจนับวงวนในกราฟทิศทาง

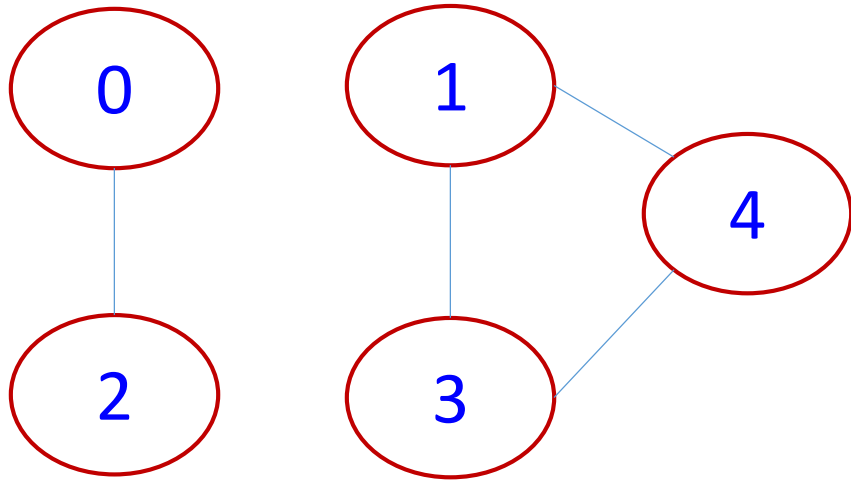
```
void DFS(vector<vector<int>>& graph, int startVertex, int V, vector<bool>& visited, vector<int>& parent)
{
    visited[startVertex] = true;

    for (int i = 0; i < V; i++) {
        if (graph[startVertex][i]) {
            if (!visited[i])
            {
                parent[i] = startVertex;
                DFS(graph, i, V, visited, parent);
            }
            else if (parent[startVertex] != i && parent[startVertex] != -1)
            {
                cycleFound = true;
                return;
            }
        }
    }
}
```

ถ้า **current vertex** มีเอดจ์ร่วมกับ **vertex i** และ **i** มีสถานะเป็น **visited** ดังนั้นหาก **i** ไม่ใช่ **parent** ของ **current vertex** จะเกิด **cycle** ในกราฟ

# การตรวจสอบ Connected Component

- ในการสำรวจกราฟด้วย DFS จะสามารถตรวจสอบได้ว่าเวอร์เท็กซ์ใดเป็นของ component เดียวกันบ้าง โดยการสำรวจความต่อเนื่อง (connectivity) ในกราฟ
- รั้น DFS เมื่อพบว่ามึบาง vertex ในกราฟมีสถานะไม่ถูก visit

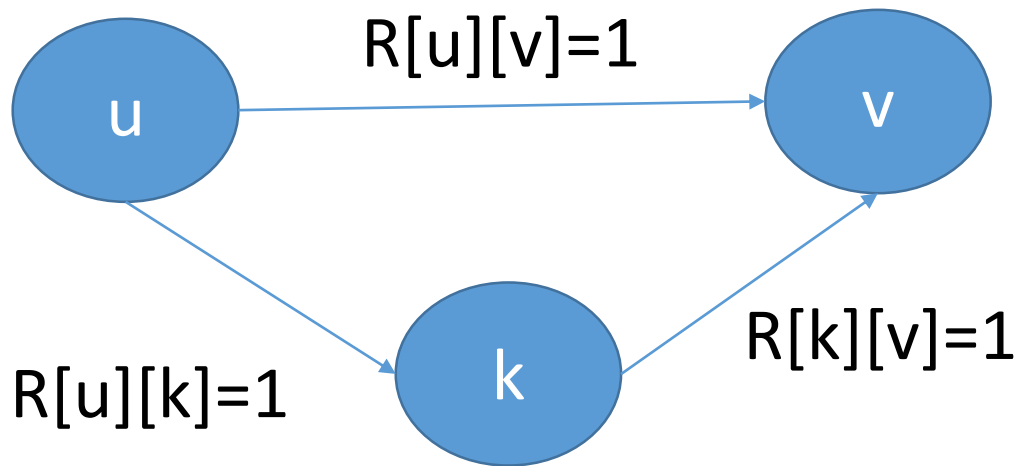


กราฟที่มี 2 connected components

```
int countgroup(int V, vector<vector<int>>& graph) {  
    vector<bool>visited(V, false);  
    int group = 0;  
    for (int i = 0; i < V; i++) {  
        if (!visited[i]) {  
            DFS(graph, i, V, visited);  
            group++;  
        }  
    }  
  
    return group;  
}
```

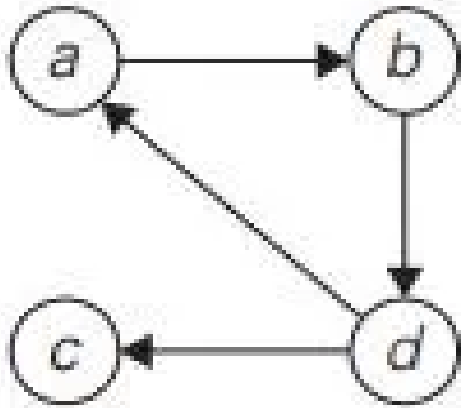
# การตรวจสอบความสัมพันธ์ระหว่างเวอร์เท็กซ์ในกราฟ

- กำหนดให้  $u$  และ  $v$  เป็นเวอร์เท็กซ์ในกราฟ เราสามารถตรวจสอบความสัมพันธ์ของ  $u$  และ  $v$  ว่า  $u$  และ  $v$  มีเอดจ์ (edge) หรือมีเส้นทางร่วมกันหรือไม่
- โดยการเช็คว่ามี edge จาก  $u$  ไป  $v$  หรือมี **transitive edges** จาก  $u$  ไป  $v$  ซึ่งมีเอดจ์รวมกันกับ intermediate vertex  $k$



$$R[u][v] = R[u][v] \vee (R[u][k] \wedge R[k][v])$$

# อัลกอริทึม Warshall



(a)

$$A = \begin{matrix} & \begin{matrix} a & b & c & d \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ d \end{matrix} & \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 \end{bmatrix} \end{matrix}$$

(b)

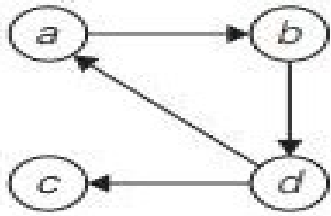
$$T = \begin{matrix} & \begin{matrix} a & b & c & d \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ d \end{matrix} & \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 \end{bmatrix} \end{matrix}$$

(c)

$$R^{(k-1)} = \begin{matrix} & \begin{matrix} j & k \end{matrix} \\ \begin{matrix} i \\ \uparrow 0 \end{matrix} & \begin{bmatrix} & \\ 1 & \\ & \\ & 1 \end{bmatrix} \end{matrix} \Rightarrow R^{(k)} = \begin{matrix} & \begin{matrix} j & k \end{matrix} \\ \begin{matrix} i \\ 1 \end{matrix} & \begin{bmatrix} & \\ 1 & \\ & 1 \end{bmatrix} \end{matrix}$$

ค้นหาเวอร์เท็กซ์ k ที่สามารถเป็น intermediate vertex ระหว่าง เวอร์เท็กซ์ i และ j

# อัลกอริทึม Warshall



$$R^{(0)} = \begin{matrix} & \begin{matrix} a & b & c & d \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ d \end{matrix} & \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 \end{bmatrix} \end{matrix}$$

$$R^{(1)} = \begin{matrix} & \begin{matrix} a & b & c & d \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ d \end{matrix} & \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 0 \end{bmatrix} \end{matrix}$$

$$R^{(2)} = \begin{matrix} & \begin{matrix} a & b & c & d \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ d \end{matrix} & \begin{bmatrix} 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 \end{bmatrix} \end{matrix}$$

$$R^{(3)} = \begin{matrix} & \begin{matrix} a & b & c & d \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ d \end{matrix} & \begin{bmatrix} 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 \end{bmatrix} \end{matrix}$$

$$R^{(4)} = \begin{matrix} & \begin{matrix} a & b & c & d \end{matrix} \\ \begin{matrix} a \\ b \\ c \\ d \end{matrix} & \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 \\ 1 & 1 & 1 & 1 \end{bmatrix} \end{matrix}$$

```

void Warshall(vector<vector<int>>& graph, int V)
{ int R[V][V];
  int i, j, k;

```

```

  for(i=0; i<V; i++)
  { for(j=0; j<V; j++)
    { R[i][j] = graph[i][j];
    }
  }

```

```

  for(k=0; k<V; k++)
  { for(i=0; i<V; i++)
    { for(j=0; j<V; j++)
      R[i][j] = R[i][j] || (R[i][k] && R[k][j]);
    }
  }

```